

Author: **Vitalii Novikov**

Degree program: **Master Artificial Intelligence & Data Science SS2026**

1st reviewer master's thesis: **Dr. Rosana De Oliveira Gomes**

github: https://github.com/vitalii-novikov/GNN_for_LOB

Results Google Sheets table: [link](#)

Real-Time Market Microstructure Modeling with Temporal Graph Neural Networks

Abstract

This thesis studies whether graph-based neural architectures improve short-horizon limit order book prediction when they are evaluated under a common, deployment-oriented protocol. The empirical setting is a cryptocurrency limit order book dataset containing ADA, BTC, and ETH snapshots at five-minute, one-minute, and one-second resolution. ETH is treated as the target asset, while ADA and BTC provide relational market context. The study compares three model families: a single-graph baseline (**base_gnn**), a multi-relation graph architecture (**multigraph**), and a stateful recurrent graph architecture (**memorygraph**). Each family is evaluated with a convolution-style graph operator and a message-passing neural network operator, producing six model variants per frequency regime and eighteen primary benchmark configurations overall.

The methodological contribution of the thesis is a controlled benchmark rather than a claim of production trading readiness. All model families share the same asset universe, feature construction, relation-state construction, triple-barrier target logic, multi-task learning objective, purged walk-forward validation design, final holdout interval, thresholding logic, and event-based backtest. This design isolates architectural differences while keeping the trading evaluation comparable. The central deployment distinction is between **last_CV**, the model state closest to a realistic deployment reference, and **final_refit**, a larger-sample refit state that is informative but not a substitute for chronological deployment evidence.

The updated empirical results show that the strongest **last_CV** model at both five-minute and one-minute frequency is **base-gnn-conv**, not the previously assumed MPNN baseline. At five-minute frequency, **base-gnn-conv** achieves $pnl_sum = 0.020356$ over 26 trades. At one-minute frequency, the same family and operator achieves $pnl_sum = 0.020094$ over 132 trades. At one-second frequency, all benchmark models remain net negative after transaction costs. The most important high-frequency finding is therefore not that no signal exists, but that gross signal and deployable net profitability diverge sharply once turnover and costs are included. The **memory-gnn-conv** model, for example, produces the largest one-second gross signal, but its very high trade count makes the net result strongly negative.

The thesis concludes that, under the controlled entry-model benchmark used here, the simpler single-graph baseline is the most reliable architecture. More complex relation-preserving and memory-based models produce useful diagnostic evidence, especially in ranking quality and gross signal extraction, but they do not establish a robust post-cost advantage. The main unresolved challenge is therefore not merely extracting short-horizon microstructure signal. It is converting that signal into sufficiently selective, stable, and cost-aware trading decisions.

Table of Contents

- [Real-Time Market Microstructure Modeling with Temporal Graph Neural Networks](#)
 - [Abstract](#)
 - [Table of Contents](#)
 - [1. Introduction](#)
 - [1.1. Motivation of the Topic](#)
 - [1.2. Research Gap and Thesis Scope](#)

- 1.3. Research Aim
- 1.4. Research Questions
- 1.5. Hypotheses
- 1.6. Thesis Contribution
- 2. Literature Background
 - 2.1. Market Microstructure and Limit Order Book Prediction
 - 2.2. Deep Learning for Limit Order Books
 - 2.3. Graph Neural Networks and Message Passing
 - 2.4. Temporal and Dynamic Graph Learning
 - 2.5. Financial Graph Learning
- 3. Data and Methodology
 - 3.1. Data Source and Study Universe
 - 3.2. Graph Input Representation
 - 3.3. Node Features
 - 3.4. Relation States and Edge Features
 - 3.5. Scaling and Leakage Control
 - 3.6. Frequency-Specific Experimental Regimes
 - 3.7. Target Construction and Shared Learning Objective
 - 3.8. Common Entry-Model Backtest
 - 3.9. Validation Design and Deployment-Oriented Model States
 - 3.10. Metrics
 - 3.11. Fair-Comparison Principle
- 4. Model Families
 - 4.1. Shared Architectural Conventions
 - 4.2. The `base_gnn` Family
 - 4.3. The `multigraph` Family
 - 4.4. The `memorygraph` Family
- 5. Results
 - 5.1. Benchmark Overview
 - 5.2. Frequency-Specific Results
 - 5.3. Answer to RQ1: Which Graph Family Performs Best?
 - 5.4. Answer to RQ2: How Important Is the Conv-versus-MPNN Operator Choice?
 - 5.5. Answer to RQ3: How Does Temporal Resolution Affect Relation and Memory Mechanisms?
 - 5.6. Answer to RQ4: Are Conclusions Stable Between `last_cv` and `final_refit`?
 - 5.7. Hypothesis Assessment
 - 5.8. Summary of Research Question Answers
 - 5.9. Summary of Hypothesis Assessment
- 6. Discussion
 - 6.1. Main Findings by Research Question
 - 6.2. Comparison with Previous Work
 - 6.3. Scientific Implications
 - 6.4. Practical and Deployment Implications
 - 6.5. Limitations, Weaknesses, and Sources of Bias
- 7. Conclusions and Future Research
 - 7.1. Overall Conclusion
 - 7.2. Future Research
- Appendix A. Model Configuration Summary
- Appendix B. Additional Benchmark Tables
- Appendix C. Final Holdout Alignment Table
- Frequency-level summary
- References

1. Introduction

1.1. Motivation of the Topic

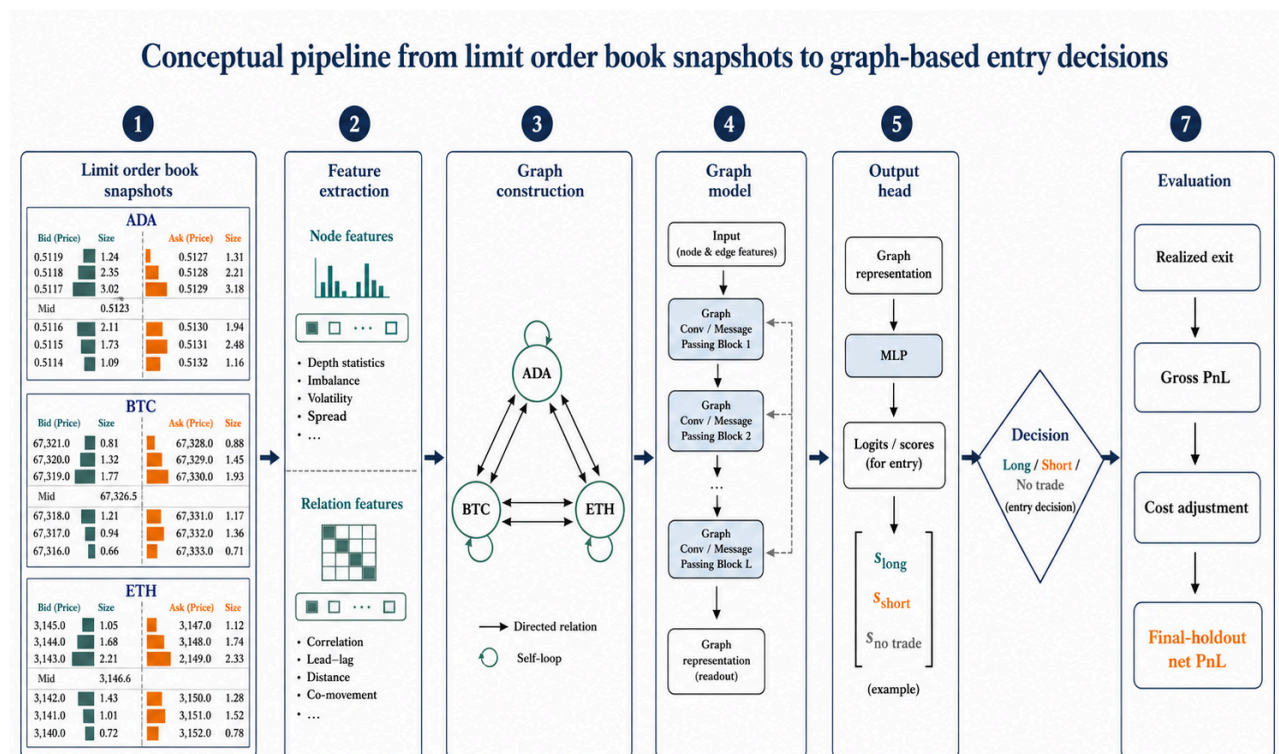
Financial markets generate large volumes of event-like information. At high frequency, prices, spreads, order-flow summaries, and visible depth change faster than a human analyst can inspect them manually. The limit order book is therefore a natural object for data science: it records the state of supply and demand near the current price and offers a dense, structured view of short-horizon market dynamics [2]. At the same time, it is one of the most difficult settings for predictive modelling. Useful signals are weak, non-stationary, highly regime-dependent, and quickly degraded by transaction costs, latency, threshold choice, and overfitting. These difficulties are consistent with well-known stylized facts of financial returns, including heavy tails, volatility clustering, and changing dependence structures [1].

This difficulty creates both a scientific and a practical motivation. Scientifically, limit order book prediction is a useful test case for machine learning on noisy sequential data. A model must process temporal dependence, cross-asset information, and changing liquidity conditions without relying on a stationary data-generating process. Practically, a forecast is not valuable only because it predicts the direction of a future move. It becomes valuable only if it can be translated into sufficiently selective trading decisions after costs. This thesis therefore treats predictive quality, gross signal quality, and net economic value as related but distinct layers of evidence.

Recent machine learning research has shown that deep architectures can extract useful representations from limit order book data. Convolutional and recurrent models have been used to learn local book structure and temporal dependencies, and large-scale studies have reported evidence that order-flow histories contain cross-instrument regularities [3-5]. These results motivate the use of representation learning, but they do not remove the need for careful evaluation. A model that produces a good ranking statistic can still fail as an entry model if it trades too frequently or if its positive gross edge is smaller than the cumulative cost burden.

Graph-based modelling provides an additional motivation. Financial assets do not evolve independently: their returns, order-flow pressure, and liquidity states can co-move, lead, lag, or diverge. A graph representation makes this relational structure explicit by representing assets as nodes and cross-asset dependencies as edges. Static graph neural networks are useful when the relation structure is fixed or slowly varying [6-9], but market microstructure is dynamic. For this reason, temporal graph neural networks, dynamic graph learning, and memory-augmented graph architectures are natural candidates for short-horizon market modelling [10-14]. Figure 1.1 summarizes the conceptual pipeline studied in the thesis.

Figure 1.1. Conceptual pipeline from limit order book snapshots to graph-based entry decisions.



Controlled research benchmark pipeline from LOB snapshots to graph-based entry decisions and cost-aware backtest evaluation.

Interim submission placeholder: This figure should show frequency-specific ADA, BTC, and ETH order book snapshots being transformed into node features, relation-aware edge states, graph neural network predictions, entry decisions, and a cost-aware final-holdout backtest.

The present thesis studies this idea in a deliberately controlled form. It does not attempt to build a full production trading system. Instead, it asks whether richer graph architectures improve a common entry-model benchmark when the data, targets, output heads, validation design, thresholding logic, and event-based trading evaluation are held as consistent as possible. The empirical question is therefore architectural: under a shared benchmark, does it help to preserve multiple relation channels, to add stateful memory, or to use a richer message-passing graph operator?

1.2. Research Gap and Thesis Scope

The literature contains several relevant strands. Market microstructure research studies how order flow, liquidity, and book depth shape short-horizon price formation. Deep learning research on limit order books shows that neural architectures can extract features from high-dimensional book states. Graph neural network research provides methods for learning from relational data. Temporal graph learning extends this idea to systems whose node states, edge states, or interaction patterns change over time. Recent financial graph studies also show that multi-relational graph structures can be useful for stock prediction at lower frequencies.

The gap addressed here is narrower and more empirical. Many financial graph studies focus on daily or lower-frequency relations, while many limit order book studies model a single instrument without explicitly representing cross-asset graph structure. This thesis examines a small but controlled crypto limit order book setting in which ADA, BTC, and ETH form a three-node graph, ETH is the target asset, and cross-asset relation states are rebuilt at **5min**, **1min**, and **1sec** resolutions. The study focuses on whether graph family, graph operator, and temporal resolution change the usefulness of relational and memory-aware modelling under a common trading-oriented evaluation.

The scope is intentionally limited. The benchmark uses a fixed asset universe, a fixed target asset, a common triple-barrier target construction, and a shared non-overlapping event backtest. This design improves internal comparability, but it also means that the thesis evaluates entry models rather than complete trading systems with jointly optimized execution and exit policies. The resulting conclusions should therefore be interpreted as evidence about architecture under a controlled benchmark, not as evidence that any model is ready for production deployment.

1.3. Research Aim

The aim of this thesis is to determine whether richer graph-based architectures improve short-horizon limit order book prediction and trading performance when evaluated under an apples-to-apples, friction-aware benchmark.

The core object of interest is the model family. The study asks whether a simple single-graph representation is sufficient, whether preserving multiple relation channels improves the result, and whether stateful memory becomes valuable at higher temporal resolution. A second object of interest is the graph operator: a Conv-style operator versus a message-passing neural network operator. A third object of interest is deployment stability: whether the same conclusions hold when moving from the last chronological cross-validation state to a final refit state.

1.4. Research Questions

The thesis is guided by four research questions. Figure 1.2 shows how the questions connect model family, graph operator, temporal resolution, and deployment-oriented model state.

RQ1. Which graph family performs best under a controlled entry-model benchmark?

The first question asks whether the simpler single-graph baseline, the multi-relation graph family, or the stateful memory graph family produces the strongest final-holdout trading result when all families are evaluated under the same target construction, thresholding logic, and event-based backtest.

RQ2. How important is the Conv-versus-MPNN operator choice inside each family?

Each family is evaluated with a Conv-style operator and an MPNN-style operator. This makes it possible to distinguish the effect of the broader family scaffold from the effect of the local graph interaction mechanism.

RQ3. How does temporal resolution change the relative value of relational and memory mechanisms?

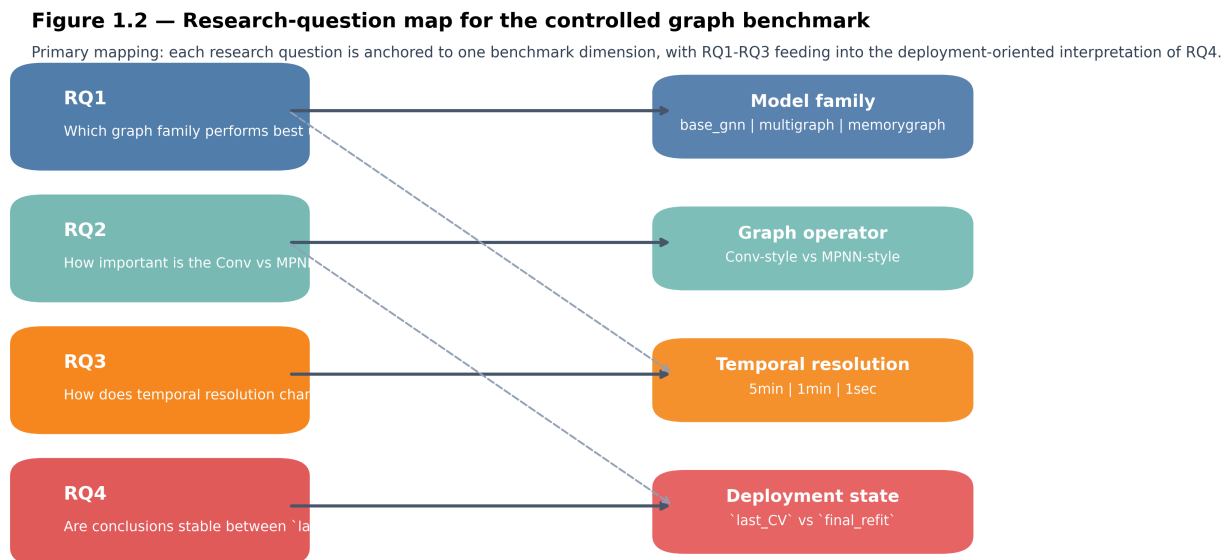
The **5min** and **1min** regimes solve the same 30-minute lookback and five-minute horizon task, while the **1sec** regime

uses a frequency-adapted two-minute lookback and two-minute horizon. This design allows the thesis to examine whether richer relation handling and recurrent memory become more useful as the observation frequency increases.

RQ4. Are the conclusions stable under deployment-oriented model states?

The thesis distinguishes between `last_CV`, the final walk-forward fold model used as the primary deployment-oriented reference, and `final_refit`, a model refit on a larger pre-holdout sample. This question asks whether the same model remains attractive when viewed through both states, and what this implies for realistic deployment interpretation.

Figure 1.2. Research-question map for the controlled graph benchmark.



Interim submission placeholder: This figure should map RQ1 to model family, RQ2 to Conv-versus-MPNN operator choice, RQ3 to temporal resolution, and RQ4 to the distinction between `last_CV` and `final_refit`.

1.5. Hypotheses

The empirical design tests five hypotheses.

H1. The one-minute regime should be the strongest shared-task benchmark.

Because the `1min` data preserve more intra-horizon dynamics than `5min` data while remaining less noisy than second-level data, the `1min` regime is expected to be the strongest of the two strict shared-task regimes.

H2. Explicit multi-relation modelling should outperform the simpler baseline more clearly at finer resolutions.

The `multigraph` family is expected to benefit from preserving price-dependence, order-flow, and liquidity channels separately, especially when cross-asset dependencies evolve quickly.

H3. Stateful memory should become more valuable as the market is observed more finely.

The `memorygraph` family is expected to be most useful at high frequency because recurrent state can, in principle, retain short-lived market information across contiguous observations.

H4. Conv and MPNN operators should not be uniformly dominant across families.

The operator comparison is expected to be family- and frequency-dependent. A Conv-style operator may be more stable when edge structure is already regularized, while an MPNN-style operator may be more useful when messages need richer source-destination-edge conditioning.

H5. `last_CV` and `final_refit` should tell a broadly consistent but not identical story.

The broad family-level conclusion is expected to remain similar across states, but individual model profitability may change when a larger pre-holdout sample is used for refitting.

1.6. Thesis Contribution

The thesis contributes a controlled empirical comparison of graph-based market microstructure models across three temporal resolutions. Its contribution is not a new universal architecture, nor a claim of production deployment readiness. Instead, it provides evidence on three narrower issues:

1. whether a simpler single-graph baseline is sufficient under a common entry-model benchmark;
2. whether explicit multi-relation handling or stateful memory improves economic outcomes after costs;
3. why deployment-oriented model states, turnover, and cost drag must be included in the interpretation of high-frequency predictive models.

From a data science perspective, the thesis contributes not only a model comparison but also a controlled evaluation workflow for noisy, non-stationary sequential data. The workflow emphasizes graph-based representation learning, leakage-aware preprocessing, chronological validation, multi-task supervised learning, cost-sensitive post-processing, and metric selection. These elements are central to the thesis because the empirical conclusion depends not only on model architecture, but also on how data are split, scaled, labelled, calibrated, and evaluated.

The main empirical finding is conservative. In the updated benchmark, `base-gnn-conv` is the strongest `last_CV` model at both `5min` and `1min`. Richer graph mechanisms sometimes improve gross signal or ranking metrics, especially at `1sec`, but these gains do not reliably translate into positive net profitability after transaction costs.

2. Literature Background

2.1. Market Microstructure and Limit Order Book Prediction

Market microstructure studies how trading rules, liquidity provision, order flow, and the organization of the limit order book affect price formation [2]. At short horizons, the visible book is informative because it contains the current distribution of buy and sell interest near the mid-price. However, short-horizon predictability is difficult to exploit. Return distributions are heavy-tailed, volatility clusters over time, and market regimes change [1]. These stylized facts make financial forecasting different from many stationary supervised-learning problems.

Limit order book modelling also creates an evaluation challenge. A direction classifier can appear useful under a conventional accuracy or AUC metric while still being economically weak if it triggers too many low-margin trades. For this reason, this thesis evaluates models through a friction-aware entry benchmark rather than through classification metrics alone. Directional AUC and trade AUC are retained as diagnostics, but `pnl_sum`, `gross_pnl_sum`, and `n_trades` are treated as central to the final interpretation.

2.2. Deep Learning for Limit Order Books

Deep learning research on limit order books has shown that neural networks can learn representations from high-dimensional book states [3]. DeepLOB is especially relevant because it combines convolutional components for local book structure with recurrent components for temporal dependence [5]. Large-scale studies of order-flow histories also suggest that neural models can identify cross-instrument regularities in price formation [4]. These studies motivate representation learning in market microstructure, but they also highlight the need for careful separation between predictive performance and economic performance.

The present thesis differs from single-instrument LOB prediction studies by making cross-asset relational structure explicit. Instead of treating ETH only as an isolated time series, ADA and BTC are included as context nodes. The resulting task is still modest in graph size, but it allows the thesis to test whether graph modelling adds value once all families share the same target and backtest.

2.3. Graph Neural Networks and Message Passing

Graph neural networks provide a general framework for learning from entities connected by relations. Graph convolutional networks and graph attention networks show how node representations can be updated using neighbourhood information, while message-passing neural networks provide a flexible formulation in which messages depend on source nodes, destination nodes, and edge attributes [6–9]. These ideas are directly relevant to financial data because assets can be represented as nodes and cross-asset dependence measures as edges.

In this thesis, the Conv-versus-MPNN distinction is used as a controlled operator comparison. Conv-style graph layers apply weighted source-node projections with edge-conditioned shifts. MPNN-style layers use richer gated messages that condition on source state, destination state, and edge state. The comparison therefore asks whether richer local message conditioning is economically useful under the same graph-family scaffold.

2.4. Temporal and Dynamic Graph Learning

Many real systems are not static graphs. Node states, edge states, and interaction patterns can evolve over time. Temporal graph networks and dynamic graph representation learning address this problem by combining graph operators with temporal encoders, memory modules, or event-driven updates [10–12]. This literature is relevant to market microstructure because cross-asset relations are unlikely to remain fixed across regimes, liquidity states, and trading intensity.

The three families in this thesis instantiate this idea at different levels of complexity. `base_gnn` uses early relation fusion and a convolutional temporal backbone. `multigraph` preserves relation-specific graph pathways longer before fusing them. `memorygraph` uses recurrent node and edge memory inside a graph-processing loop. The empirical question is not whether these mechanisms are theoretically plausible; it is whether they improve a controlled friction-aware benchmark.

2.5. Financial Graph Learning

Financial applications of graph neural networks include stock relation modelling, portfolio prediction, risk propagation, and transaction-network analysis [13]. Recent multi-relational dynamic graph work is especially relevant because financial relations can be heterogeneous: firms, sectors, correlations, supply chains, and market co-movements can all define different relation channels [14]. This thesis adopts a microstructure version of the same general idea by constructing relation channels from price dependence, order-flow dependence, and liquidity dependence.

However, this thesis remains cautious about novelty. It does not claim to solve dynamic financial graph learning in general. Its contribution is an internally controlled comparison on a specific crypto LOB dataset and a specific entry-model benchmark.

The literature therefore motivates the empirical design that follows. Market microstructure explains why the prediction problem is noisy and friction-sensitive; deep LOB models motivate neural representation learning; GNN and temporal graph methods motivate cross-asset and memory-aware architectures; and financial graph learning motivates relation channels. The next chapter translates these ideas into the controlled data representation and evaluation protocol used in the thesis.

3. Data and Methodology

3.1. Data Source and Study Universe

The raw data source is the public Kaggle dataset *High-Frequency Crypto Limit Order Book Data* by Martinsn, which provides frequency-specific cryptocurrency limit order book snapshots for multiple assets, including ADA, BTC, and ETH, at `1sec`, `1min`, and `5min` resolutions [15]. The data are distributed as order book snapshots organized by price level rather than as raw exchange message streams.

The present study uses a fixed three-node asset universe:

1. ADA
2. BTC
3. ETH

ETH is the target asset. ADA and BTC provide relational market context. Because the source data are already distributed in frequency-specific tables, no bespoke reconstruction of the limit order book from raw order messages is required. The preprocessing task is instead to standardize timestamps, align assets on a common clock, and derive node and edge features from the available order book summaries.

The local data files used in the pipelines contain midpoint price, spread, buy and sell flow summaries, and 15 bid-side and 15 ask-side depth values. These fields are the foundation for all node features and relation features used in the benchmark.

3.2. Graph Input Representation

All models use the same graph input representation within a frequency regime. The graph is a directed complete graph over the three assets with self-loops. The nodes are fixed, but node states and edge states vary over time.

Formally, each model receives:

1. a node sequence $X^{\{(n)\}} \in \mathbb{R}^{B \times L \times N \times F_n}$
2. a relation-aware edge sequence $X^{\{(e)\}} \in \mathbb{R}^{B \times L \times R \times E \times F_e}$

where B is batch size, L is the lookback length, $N = 3$ is the number of assets, $R = 3$ is the number of relation channels, and E is the number of directed edges including self-loops.

The three relation channels are:

1. **price_dep**, based on asset log returns.
2. **order_flow**, based on flow imbalance scaled by log turnover.
3. **liquidity**, based on spread, depth imbalance, near-depth imbalance, and near/far depth shape.

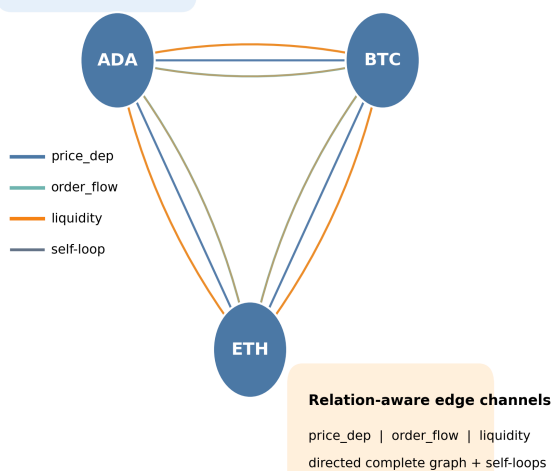
The overall graph representation is summarized in Figure 3.1.

Figure 3.1. Graph input representation for the three-asset limit order book benchmark.

Figure 3.1 — Graph input representation for the three-asset limit order book benchmark

Node features (per asset, per time step)

returns | spread | buys | sells | flow imbalance
depth imbalance | top-level imbalance | near/far depth shape



Three assets are modeled as nodes in a directed complete graph; each edge carries multiple relation channels and each node carries microstructure features extracted from its own order book.

*Interim submission placeholder: This figure should illustrate ADA, BTC, and ETH as nodes in a directed complete graph with self-loops and relation-aware edges for **price_dep**, **order_flow**, and **liquidity**.*

3.3. Node Features

For each asset and each time step, the node feature block summarizes local price behavior, order-flow pressure, and depth structure. The implemented node features are:

1. one-bar log return.
2. relative spread.
3. log-transformed buys.
4. log-transformed sells.

5. flow imbalance.
6. total depth imbalance.
7. top-level depth imbalances for the first five book levels.
8. bid near/far depth ratio.
9. ask near/far depth ratio.
10. near-depth imbalance.
11. far-depth imbalance.

This feature set is deliberately microstructure-oriented. It does not use external news, social media, or macroeconomic variables. That choice keeps the thesis focused on the information available inside the aligned cross-asset order book state.

3.4. Relation States and Edge Features

Edge features are constructed from rolling cross-asset dependence measures. For every ordered asset pair and every relation channel, the pipeline computes lagged rolling features over frequency-specific windows:

1. rolling correlation.
2. rolling beta.
3. rolling mean product.

When configured, rolling correlations are Fisher-\$z\$ transformed before scaling. The edge tensor therefore represents relation-specific dependence among assets rather than only a fixed adjacency prior.

This design is important for fair comparison. All three model families operate on the same handcrafted relation states and the same learnable pairwise edge-fusion path. The architectures differ in how they process and fuse this information, not in whether they receive richer or poorer input data.

3.5. Scaling and Leakage Control

Node and edge tensors are robustly scaled on training data only, using fold-specific quantile statistics. The transformed features are then clipped to bounded ranges before model fitting. This prevents train-test leakage through scaling and reduces the influence of extreme observations. Because the same scaling approach is used for all families, feature preprocessing does not favor any architecture.

3.6. Frequency-Specific Experimental Regimes

The experimental design contains eighteen primary runs: six model variants for each of the three frequency regimes.

The **5min** and **1min** regimes solve the same clock-time task:

1. lookback window = 30 minutes.
2. forecast horizon = 5 minutes.

This corresponds to six lookback bars and one horizon bar at **5min**, and 30 lookback bars and five horizon bars at **1min**.

The **1sec** regime uses a frequency-adapted task:

1. lookback window = 2 minutes = 120 bars.
2. forecast horizon = 2 minutes = 120 bars.

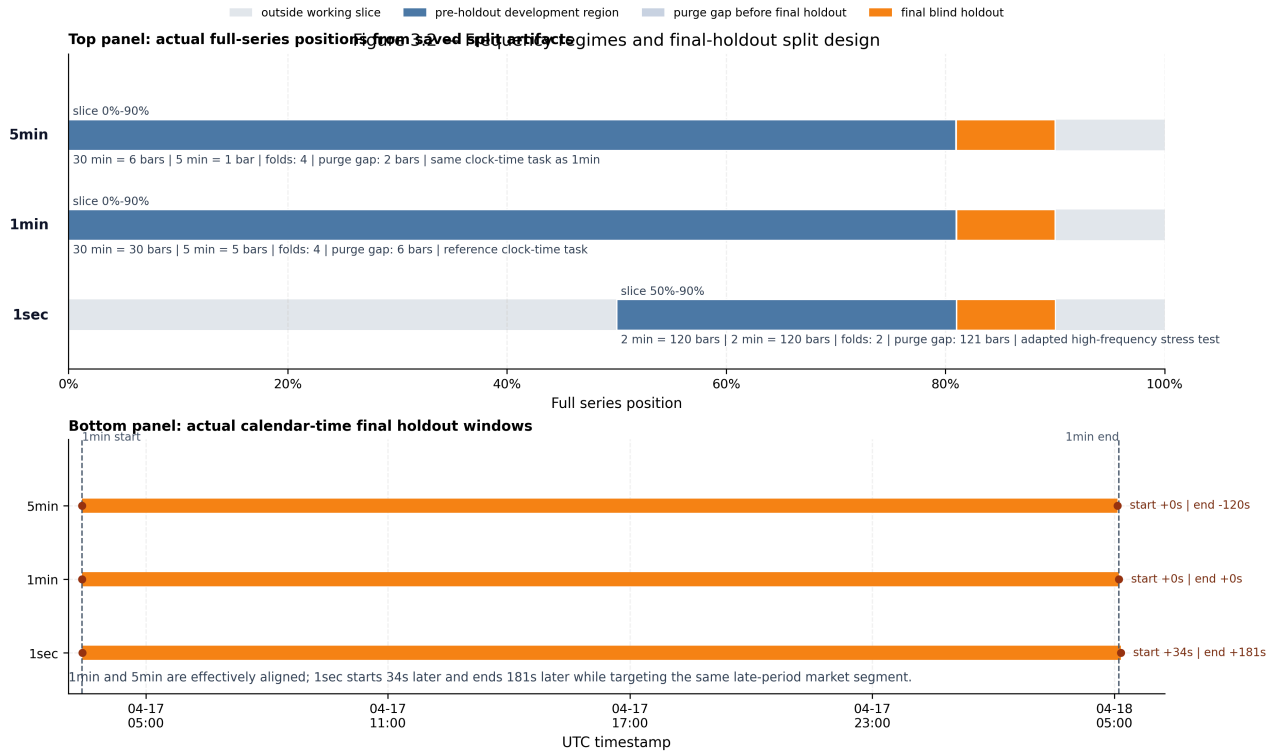
The one-second working sample is restricted to the interval from 50% to 90% of the full second-level series. This keeps training computationally feasible while preserving a late-period high-frequency comparison. The final holdout fraction is increased to align the one-second blind evaluation interval as closely as possible with the final holdout interval used in the slower-frequency experiments.

Table 3.1. Frequency-specific experimental regimes and validation settings.

Frequency	Working data slice	Final holdout fraction	Lookback	Horizon	CV folds
5min	0.0–0.9 of the full series	0.10	30 min = 6 bars	5 min = 1 bar	4
1min	0.0–0.9 of the full series	0.10	30 min = 30 bars	5 min = 5 bars	4
1sec	0.5–0.9 of the full series	0.225	2 min = 120 bars	2 min = 120 bars	2

The frequency-specific sample design is shown schematically in Figure 3.2.

Figure 3.2. Frequency regimes and final-holdout split design.



Interim submission placeholder: This figure should compare the 5min, 1min, and 1sec working samples, the pre-holdout region, purge-aware cross-validation folds, and the aligned final holdout intervals.

The 5min and 1min regimes are therefore directly comparable as a strict shared-task benchmark. The 1sec regime is apples-to-apples within its own frequency, but it should be interpreted as a frequency-adapted high-frequency stress test rather than as a perfectly symmetric continuation of the 30-minute lookback and five-minute horizon task.

3.7. Target Construction and Shared Learning Objective

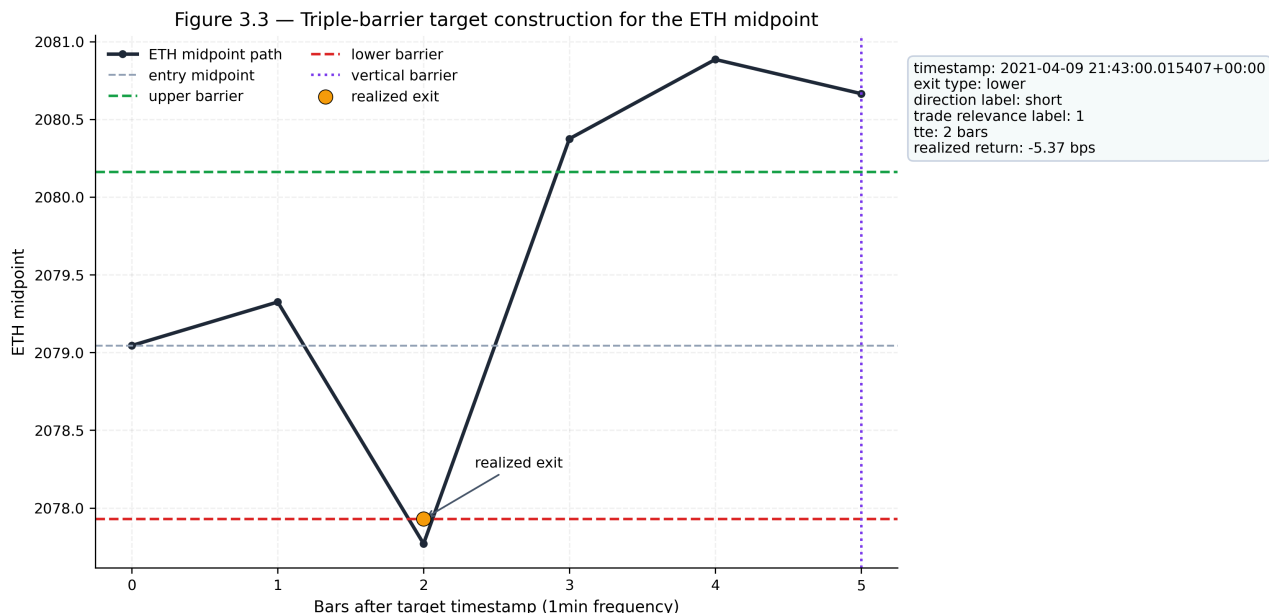
All model families are trained under the same multi-task triple-barrier framework. For each valid timestamp t , the future path of the ETH midpoint is followed until one of three mutually exclusive events occurs:

1. the upper barrier is touched.
2. the lower barrier is touched.
3. the vertical barrier is reached.

The barrier system is volatility-scaled. In the default benchmark configuration, the upper and lower barriers start from 8 basis points, are rescaled using rolling volatility estimated over a 30-bar lookback, are multiplied by 1.8, and are clipped to the interval from 4 to 30 basis points. The vertical barrier is set equal to the prediction horizon.

Figure 3.3 summarizes the triple-barrier target construction used before the common target set is derived.

Figure 3.3. Triple-barrier target construction for the ETH midpoint.



Selection rule: first 1min ETH example after the 30-bar volatility warm-up with a trade-relevant non-vertical event and a time-to-exit closest to the middle of the 5-bar horizon.

Interim submission placeholder: This figure should show the upper barrier, lower barrier, vertical barrier, realized event exit, realized return, trade relevance label, and direction label for one target timestamp.

From this future path, the pipeline constructs a common target set:

1. realized return.
2. trade relevance label.
3. direction label.
4. exit-type label.
5. time-to-exit label.

The trade label is meta-labeled and depends on whether the future move remains economically meaningful after a friction-aware threshold is applied. Direction labels are masked when timeout outcomes are configured as uninformative for directional supervision.

All families share the same output interface:

1. `trade_logit`
2. `dir_logit`
3. `return_pred`
4. `exit_type_logit`
5. `tte_pred`

The multi-task objective combines trade classification, direction classification, return regression, utility-based supervision, exit-type classification, and time-to-exit regression. In the benchmark configuration, the loss weights are:

1. `loss_w_trade = 0.35`
2. `loss_w_dir = 0.65`
3. `loss_w_ret = 0.15`
4. `loss_w_utility = 0.85`
5. `loss_w_exit_type = 0.05`
6. `loss_w_tte = 0.03`

This shared target design preserves comparability. The models differ in how they encode temporal and graph structure, not in what they are asked to predict.

3.8. Common Entry-Model Backtest

The trading evaluation is formulated as a common entry-model benchmark. In the primary backtest:

1. the trade head determines whether a trade candidate is active.
2. the direction head determines whether the candidate becomes a long or short position.
3. the exit is generated by the same realized event rule for all families.

Exit-type and time-to-exit heads are retained as auxiliary learning targets and diagnostics, but they do not define a family-specific trade-closing policy in the main benchmark. This choice is especially important for **memorygraph**, because a stateful architecture could otherwise be evaluated under a different execution policy from the other families. The common entry-model benchmark improves internal validity by holding execution logic fixed.

The trading evaluation uses a sequential non-overlapping event-based backtest. Once a position is opened, no new position can be opened until the current one is closed. This makes turnover interpretable and avoids overlapping position exposure.

For trade i , gross PnL is computed as:

$$\text{gross_pnl}_i = s_i \cdot r_i,$$

where $s_i \in \{-1, +1\}$ is the trade side and r_i is the realized log return up to the realized event exit. Net PnL is:

$$\text{net_pnl}_i = \text{gross_pnl}_i - c_{\text{rt}},$$

where the round-trip transaction-cost proxy is:

$$c_{\text{rt}} = 3 \times \text{cost_bps_per_side} \times 10^{-4}.$$

With $\text{cost_bps_per_side} = 1.0$, this gives:

$$c_{\text{rt}} = 0.0003.$$

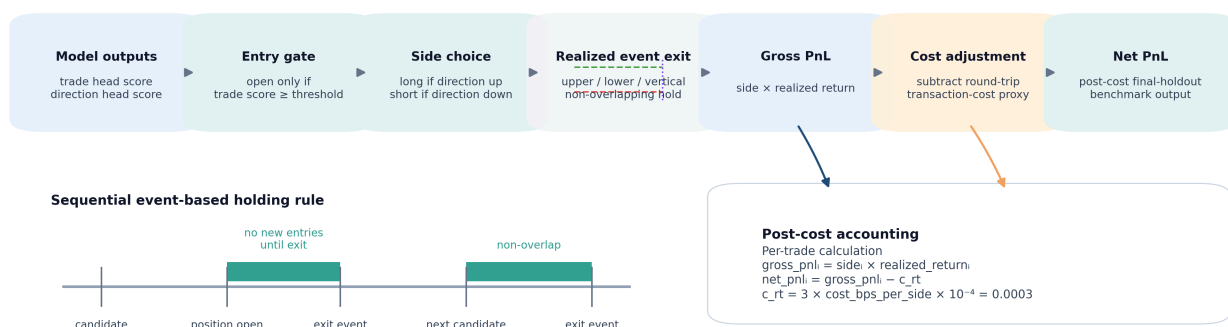
The cost model is deliberately simple. It is sufficient for a controlled friction-aware benchmark, but it should not be interpreted as a complete execution simulator.

The common entry-model evaluation logic is illustrated in Figure 3.4.

Figure 3.4. Common entry-model backtest and post-cost PnL calculation.

Figure 3.4 — Common entry-model backtest and post-cost PnL calculation

Controlled entry-model evaluation: a model may open a candidate position, the realized event closes it, and costs convert gross signal into deployable net evidence.



Scope note: the diagram documents the common controlled backtest used for thesis comparison; It is not a live execution simulator.

Interim submission placeholder: This figure should show how the trade head activates a candidate, the direction head selects long or short exposure, the realized event rule closes the position, and gross PnL is converted to net PnL after the transaction-cost proxy.

3.9. Validation Design and Deployment-Oriented Model States

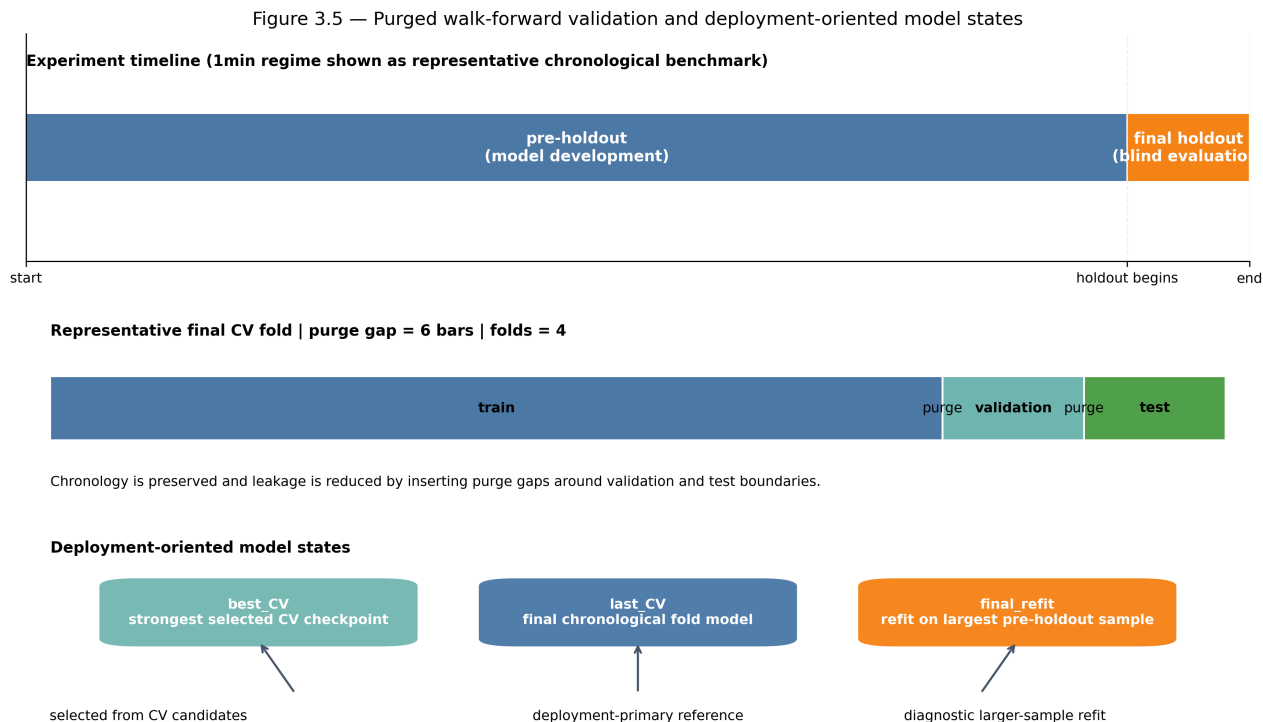
The experiments use purged walk-forward validation. Each working sample is divided into:

1. a pre-holdout region used for model development.
2. a final holdout region used only for blind final evaluation.

Within the pre-holdout region, each walk-forward fold follows a chronological train-gap-validation-gap-test structure. The purge gaps are necessary because triple-barrier labels depend on future price evolution; adjacent observations can have overlapping future windows and would otherwise leak information across split boundaries.

The purged walk-forward design is shown schematically in Figure 3.5.

Figure 3.5. Purged walk-forward validation and deployment-oriented model states.



*Interim submission placeholder: This figure should show chronological train, purge gap, validation, purge gap, test, and final-holdout segments, and should indicate how **best_CV**, **last_CV**, and **final_refit** are produced.*

The study distinguishes three model states:

1. **best_CV**, the strongest selected cross-validation checkpoint.
2. **last_CV**, the model from the last chronological walk-forward fold.
3. **final_refit**, the model refit on the largest possible pre-holdout sample.

The main thesis benchmark uses **last_CV**. This state is the most deployment-relevant reference because it approximates a model selected from the most recent chronological validation cycle before the final holdout. The **final_refit** state adds a useful larger-sample comparison, but it cannot replace **last_CV**: refitting changes the training sample, may change the score-to-trade conversion, and is not itself evidence that the same model would have been selected in a live walk-forward process.

3.10. Metrics

The main empirical metrics are:

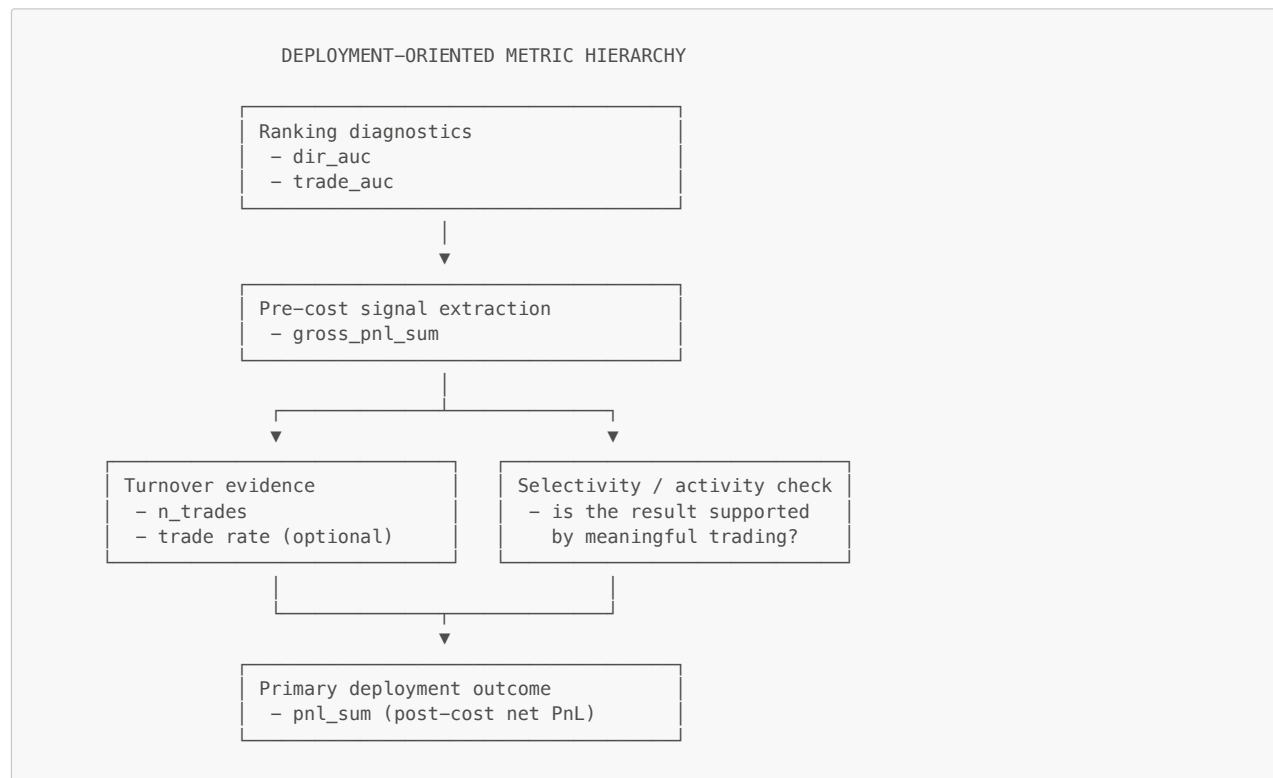
1. **gross_pnl_sum**, the sum of pre-cost directional trade returns.
2. **pnl_sum**, the sum of post-cost trade returns.
3. **n_trades**, the number of executed trades.
4. **dir_auc**, the AUC of the direction head.
5. **trade_auc**, the AUC of the trade head.

The primary economic metric is `pnl_sum`. The `gross_pnl_sum` metric separates raw signal extraction from the effect of transaction costs. The `n_trades` metric shows whether the result is supported by meaningful trading activity or by a small number of positions. The AUC metrics are valuable diagnostics for ranking quality, but they are not sufficient evidence of deployable profitability.

The interpretation hierarchy for these metrics is summarized in Figure 3.6.

Figure 3.6. Metric hierarchy for deployment-oriented interpretation.

Interim submission placeholder: This figure should present `dir_auc` and `trade_auc` as ranking diagnostics, `gross_pnl_sum` as pre-cost signal extraction, `n_trades` as turnover evidence, and `pnl_sum` as the primary post-cost economic outcome.



3.11. Fair-Comparison Principle

Within each frequency regime, only two aspects are allowed to vary:

1. the family scaffold (`base_gnn`, `multigraph`, `memorygraph`).
2. the local graph operator (Conv or MPNN).

The following elements are held fixed within a regime:

1. asset universe and target asset.
2. node-feature construction.
3. relation-state construction.
4. edge-feature construction.
5. label construction.
6. multi-task output interface.
7. thresholding logic.
8. event-based backtest.
9. split protocol.
10. final holdout interval.

This is the methodological basis for treating the benchmark as an architecture comparison rather than as a comparison of unrelated trading systems.

4. Model Families

4.1. Shared Architectural Conventions

All three model families operate on the same node and edge tensors and produce the same multi-task outputs. They also share a hybrid edge-fusion mechanism that augments handcrafted relation features with learnable pairwise node interactions. The main architectural differences concern:

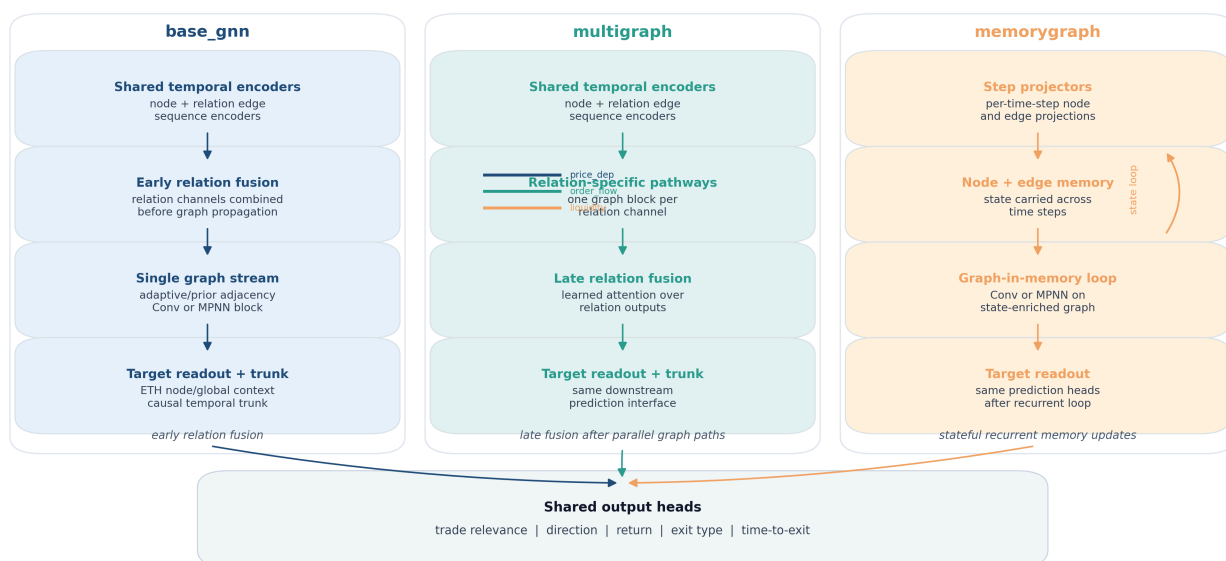
1. when relation channels are fused.
2. whether the temporal backbone is convolutional or recurrent.
3. whether the local graph operator is Conv-style or MPNN-style.

The architectural comparison is summarized in Figure 4.1.

Figure 4.1. Architecture comparison of `base_gnn`, `multigraph`, and `memorygraph`.

Figure 4.1 — Architecture comparison of `base_gnn`, `multigraph`, and `memorygraph`

All families consume the same graph tensors and emit the same multi-task heads; the contrast is relation fusion timing and recurrent memory.



Code-grounded anchors: `BaseTemporalFusionModel`, `MultigraphTemporalFusionModel`, and `MemoryGraphTemporalFusionModel` share heads but differ in relation fusion and memory state.

Interim submission placeholder: This figure should compare early relation fusion in `base_gnn`, relation-specific graph pathways in `multigraph`, and recurrent node-edge memory updates in `memorygraph`, while showing that all families share the same output heads.

The two local graph operator types can be summarized as follows. The Conv-style operator applies a weighted source-node projection plus an edge-conditioned shift term. The MPNN-style operator computes gated messages conditioned on source node state, destination node state, and edge state. This makes the MPNN operator more expressive, but not automatically more profitable.

4.2. The `base_gnn` Family

The `base_gnn` family is the single-graph baseline. It is evaluated through two adaptive operators, corresponding to the benchmark variants `base-gnn-conv` and `base-gnn-mpnn`:

1. `adaptive_conv`
2. `adaptive_mpnn`

The temporal component is fully convolutional. Node inputs are projected into hidden space, augmented with learned asset embeddings, and processed by dilated causal residual convolution blocks. Edge inputs are processed by a separate temporal edge encoder. After graph processing and readout, the target-centered sequence is passed through a second causal temporal trunk.

The graph component first fuses relation-aware edge features, then collapses the relation axis into a single edge representation. A single graph operator block is then applied using adaptive adjacency. This means that **base_gnn** tests whether an early-fused relation representation is sufficient for the benchmark.

The readout concatenates the target-node representation with global graph context, including mean and max pooling and optional target-to-global attention. The resulting target-centered representation is mapped to the shared multi-task prediction heads.

4.3. The **multigraph** Family

The **multigraph** family extends the baseline by preserving relation channels deeper into the graph-processing stage. It is evaluated in two matched variants, corresponding to **multi-gnn-conv** and **multi-gnn-mpnn**:

1. **dynamic_rel_conv**
2. **dynamic_edge_mpnn**

The temporal component is structurally similar to **base_gnn**: node and edge histories are encoded with dilated causal convolution blocks, and the target readout is processed by a causal temporal trunk. The difference is in graph processing. Instead of collapsing the relation axis before message passing, the model constructs a separate relation graph block for each relation channel.

For each relation, the Conv variant computes dynamic edge scores and applies normalized source-node projections and edge-conditioned shifts. The MPNN variant uses gated messages conditioned jointly on source state, destination state, and edge state. After relation-specific processing, the model applies learned relation attention fusion.

The central design question for **multigraph** is whether preserving price-dependence, order-flow, and liquidity relations as separate graph pathways improves the final trading benchmark relative to early relation fusion.

4.4. The **memorygraph** Family

The **memorygraph** family is the most distinct architecture in the study. It is evaluated with two variants, **memory-gnn-conv** and **memory-gnn-mpnn**:

1. **conv**
2. **mpnn**

Unlike **base_gnn** and **multigraph**, it does not rely on a deep causal-convolutional temporal encoder. Instead, it uses stateful recurrent memory. Raw node and edge inputs are first projected at each time step. A **MemoryAugmentedGraphBlock** then maintains node memory and relation-specific edge memory across contiguous chunks.

The edge memory update uses recurrent cells conditioned on current edge state, source-node state, destination-node state, and pairwise node interactions. The node memory update aggregates relation-specific edge-memory context to nodes, fuses relation-specific node and edge contexts, and updates node memory with another recurrent cell. Training uses contiguous stateful chunks with truncated backpropagation through time.

Inside each recurrent step, the graph operator is either Conv-style or MPNN-style. The key difference from the other families is that graph interaction occurs inside a recurrent memory loop. The operator acts on state-enriched representations rather than on a fully pre-encoded temporal sequence.

This gives **memorygraph** a qualitatively different inductive bias:

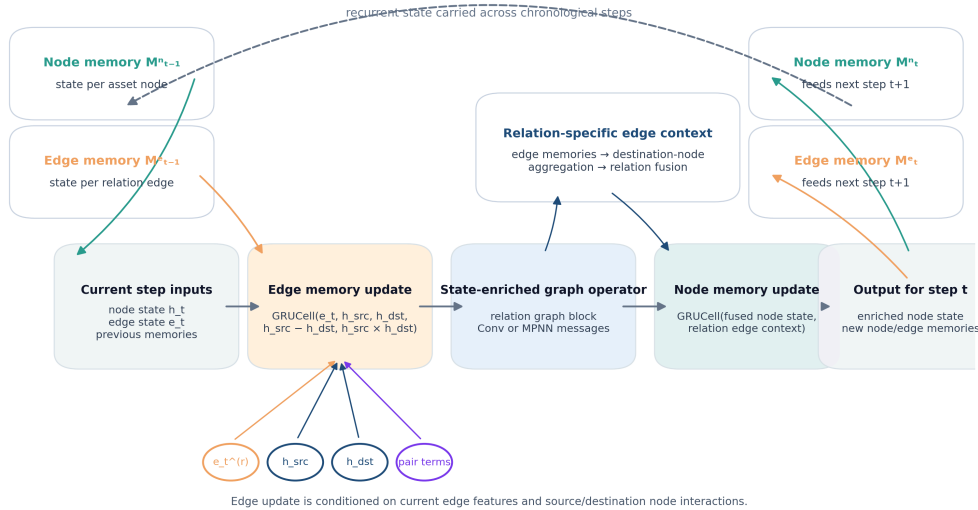
1. **base_gnn** uses early relation fusion and convolutional temporal modelling.
2. **multigraph** uses late relation fusion and convolutional temporal modelling.
3. **memorygraph** uses relation-aware recurrent state and stateful graph updates.

Figure 4.2 highlights the recurrent memory mechanism that differentiates **memorygraph** from the convolutional temporal families.

Figure 4.2. Recurrent node and edge memory update in **memorygraph.**

Figure 4.2 — Recurrent node and edge memory update in memorygraph

Memorygraph differs from the convolutional temporal families by updating node and edge states inside a recurrent graph loop at each time step.



Code-grounded anchors: EdgeMemoryUpdater uses GRUCell(edge state + source/destination pair features); NodeMemoryUpdater aggregates relation edge memory to nodes and updates node memory with another GRUCell.

Interim submission placeholder: This figure should show edge memory updates conditioned on current edge state, source node state, destination node state, and pairwise node interactions, followed by node memory updates from relation-specific edge contexts.

5. Results

This chapter reports the empirical benchmark. The main evidence is the deployment-oriented **last_CV** comparison across all eighteen primary model-frequency configurations. The chapter then discusses frequency-specific outcomes, answers the research questions, compares selected **last_CV** and **final_refit** cases, and evaluates the hypotheses.

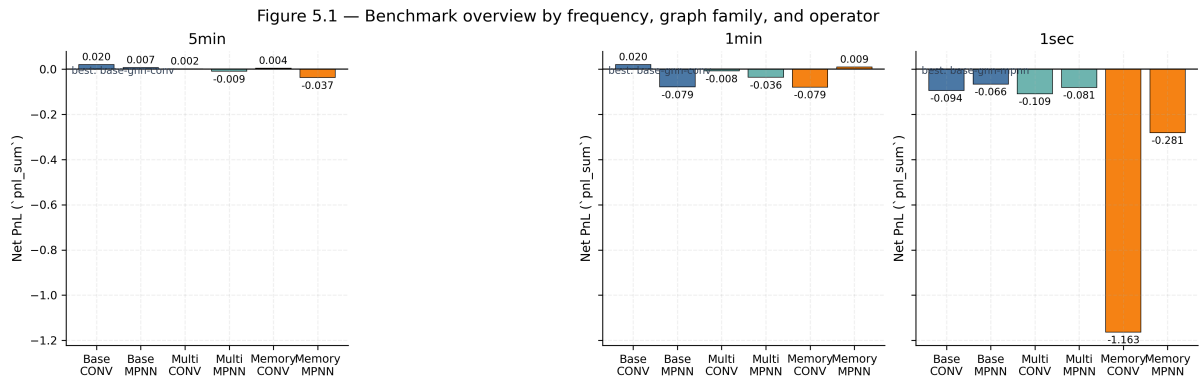
The main interpretive rule is that **pn_l_sum** is the primary economic outcome, **gross_pn_l_sum** indicates pre-cost signal extraction, and **n_trades** is necessary for understanding whether the economic result is operationally meaningful. AUC values are interpreted as ranking diagnostics, not as sufficient evidence of tradability.

5.1. Benchmark Overview

Table 5.1 reports the updated **last_CV** benchmark. Within each frequency, the six models are directly comparable because they use the same input representation, target construction, validation logic, and event-based backtest. The **5min** and **1min** regimes are also directly comparable to each other because they solve the same 30-minute lookback / five-minute horizon task. The **1sec** regime should be interpreted as a high-frequency stress test with its own adapted task.

Figure 5.1 provides a suggested visual summary of the benchmark grid before the exact numerical values are reported in Table 5.1.

Figure 5.1. Benchmark overview by frequency, graph family, and operator.



Note: visual summary of primary last_CV benchmark results; no uncertainty intervals or formal dominance testing are shown.

Interim submission placeholder: This figure should visualize `pnL_sum` for all eighteen `last_CV` model-frequency configurations, grouped by frequency and by graph family/operator.

Table 5.1. `last_CV` benchmark overview across all model-frequency configurations.

Frequency	Model	gross_pnl_sum	pnL_sum	n_trades	dir_auc	trade_auc
5min	base-gnn-conv	0.028156	0.020356	26	0.617105	0.700447
5min	base-gnn-mpnn	0.014415	0.006915	25	0.614912	0.727631
5min	multi-gnn-conv	0.012758	0.001958	36	0.616667	0.672097
5min	multi-gnn-mpnn	0.002941	-0.009359	41	0.625439	0.707304
5min	memory-gnn-conv	0.009459	0.004359	17	0.611842	0.734196
5min	memory-gnn-mpnn	-0.012463	-0.037363	83	0.537719	0.726026
1min	base-gnn-conv	0.059694	0.020094	132	0.525927	0.648157
1min	base-gnn-mpnn	-0.024239	-0.078539	181	0.504512	0.642097
1min	multi-gnn-conv	0.025767	-0.007833	112	0.541996	0.634787
1min	multi-gnn-mpnn	-0.003371	-0.035771	108	0.528405	0.645004
1min	memory-gnn-conv	-0.031247	-0.078947	159	0.479399	0.638928
1min	memory-gnn-mpnn	0.033605	0.009305	81	0.529844	0.635665
1sec	base-gnn-conv	0.139515	-0.094185	779	0.599753	0.841804
1sec	base-gnn-mpnn	0.052679	-0.065821	395	0.599093	0.848558
1sec	multi-gnn-conv	0.081710	-0.108790	635	0.597838	0.839544
1sec	multi-gnn-mpnn	0.079777	-0.080723	535	0.600538	0.868370
1sec	memory-gnn-conv	0.412032	-1.163268	5251	0.588785	0.490050
1sec	memory-gnn-mpnn	0.223788	-0.280512	1681	0.596713	0.863699

Three patterns define the updated benchmark.

First, `base-gnn-conv` is the strongest model at both `5min` and `1min`. This is the most important correction relative to the earlier interpretation of the results. The benchmark no longer supports the claim that `base-gnn-mpnn` is the strongest model at the shared-task frequencies. The updated evidence favors the Conv variant of the baseline family.

Second, all one-second models remain net negative after transaction costs. Several of them produce positive gross signal, and some produce large gross signal, but none converts that signal into positive `pnL_sum`. The one-second result should therefore not be interpreted as a search for a small positive winner. It is a demonstration that gross signal and net deployability separate sharply under high turnover.

Third, richer architectures do not dominate the primary economic benchmark. **multigraph** sometimes improves ranking diagnostics or gross PnL, and **memorygraph** produces the largest gross signal at one-second frequency. Nevertheless, neither family establishes a robust post-cost advantage over the simpler **base_gnn** scaffold under the common entry-model evaluation.

The results should be interpreted as controlled empirical benchmark evidence rather than as a formal statistical dominance test. The thesis reports final-holdout outcomes and selected deployment-state comparisons, but it does not include repeated-seed uncertainty, bootstrap confidence intervals, or pairwise significance tests for economic metrics. This limitation does not invalidate the architecture comparison, but it means that small differences, especially between the best **5min** and **1min** net outcomes, should be interpreted cautiously.

5.2. Frequency-Specific Results

5.2.1. Five-Minute Regime

The **5min** regime produces the clearest economically positive block of results. The best model is **base-gnn-conv**, with **gross_pnl_sum** = 0.028156, **pnl_sum** = 0.020356, and 26 trades. The second-best model is **base-gnn-mpnn**, with **pnl_sum** = 0.006915 over 25 trades. The two baseline variants therefore occupy the top two economic positions.

The more complex families remain informative but not dominant. **multi-gnn-conv** is mildly positive, with **pnl_sum** = 0.001958, while **multi-gnn-mpnn** is negative. **memory-gnn-conv** is also mildly positive, with **pnl_sum** = 0.004359, while **memory-gnn-mpnn** is the weakest five-minute model with **pnl_sum** = -0.037363 and 83 trades.

The ranking metrics show why economic interpretation cannot rely on AUC alone. **multi-gnn-mpnn** has the highest **dir_auc** in the five-minute block (0.625439), and **memory-gnn-conv** has the highest **trade_auc** (0.734196). Neither is the best economic model. The best deployable outcome comes from the baseline Conv model, which combines a positive gross signal with a modest number of trades and limited cost drag.

5.2.2. One-Minute Regime

The **1min** regime is the richer shared-task stress test because it uses the same 30-minute lookback and five-minute horizon as the five-minute regime, but with more granular input information. The winner remains **base-gnn-conv**, with **gross_pnl_sum** = 0.059694, **pnl_sum** = 0.020094, and 132 trades.

This result is important because the gross signal is much larger than at five-minute frequency, but the net result is almost identical. The reason is turnover. The one-minute model extracts more pre-cost signal, but the larger number of trades absorbs most of the incremental edge through transaction costs. The one-minute benchmark is therefore not stronger in strict net-profit terms, but it is stronger as a stress test of whether signal survives more active trading.

The second-best one-minute model is **memory-gnn-mpnn**, with **pnl_sum** = 0.009305 over 81 trades. This is the strongest shared-task result for **memorygraph** and suggests that recurrent memory can be useful at minute-level resolution. However, the result remains below the baseline Conv winner.

The **multigraph** family does not produce positive net PnL at one-minute frequency. **multi-gnn-conv** has positive gross PnL (0.025767) but ends at **pnl_sum** = -0.007833; **multi-gnn-mpnn** is also negative. This does not show that relation-specific modelling contains no information. It shows that, under the present thresholding and cost assumptions, relation-specific information does not translate into superior net profitability.

5.2.3. One-Second Regime

The **1sec** regime creates the sharpest separation between gross signal and net deployability. All models finish negative on **pnl_sum**. The least negative model is **base-gnn-mpnn** at **pnl_sum** = -0.065821, followed by **multi-gnn-mpnn** at -0.080723, **base-gnn-conv** at -0.094185, and **multi-gnn-conv** at -0.108790. The memory models are substantially more negative.

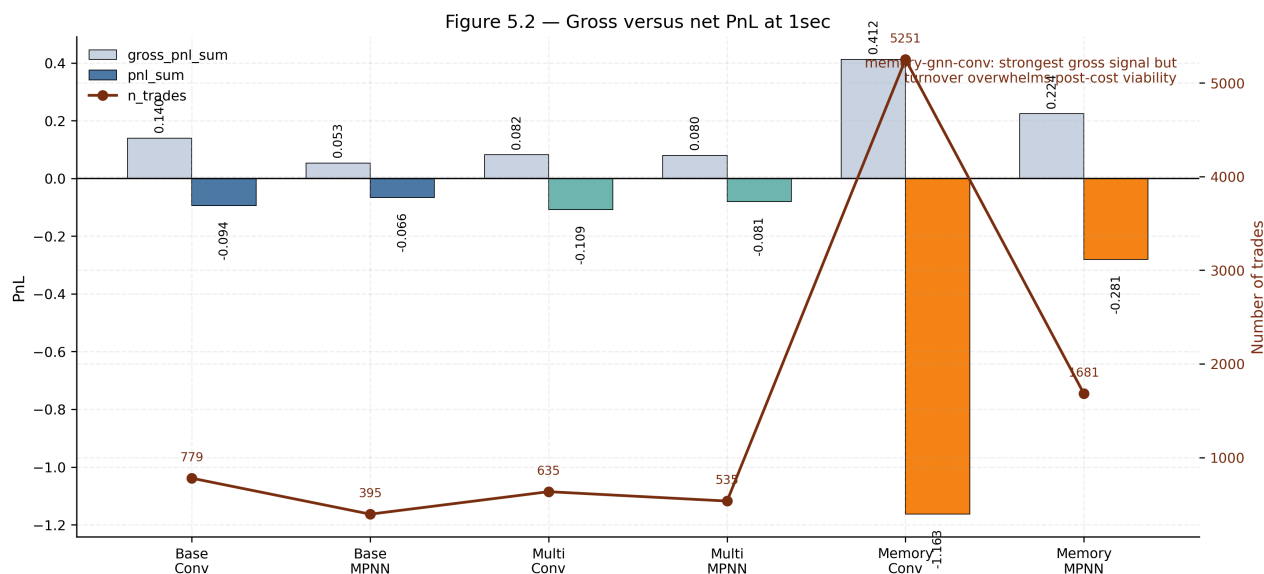
The gross results tell a different story. **memory-gnn-conv** produces the largest gross signal in the entire benchmark (**gross_pnl_sum** = 0.412032), and **memory-gnn-mpnn** produces the second-largest one-second gross signal (0.223788). These numbers indicate that the memory architecture is extracting high-frequency structure. However,

`memory-gnn-conv` executes 5251 trades and ends at `pnl_sum` = -1.163268 ; `memory-gnn-mpnn` executes 1681 trades and ends at `pnl_sum` = -0.280512 .

At the benchmark cost of 0.0003 per trade, the cumulative cost burden for `memory-gnn-conv` is approximately 1.5753 . This cost burden is much larger than its gross signal of 0.412032 . The model is therefore not failing because it lacks raw signal. It is failing because the signal is expressed through too many trades.

Figure 5.2 highlights this divergence between gross and net PnL in the one-second regime.

Figure 5.2. Gross versus net PnL at 1sec.



Interim submission placeholder: This figure should compare `gross_pnl_sum` and `pnl_sum` for the six 1sec models and annotate the role of `n_trades`, especially for `memory-gnn-conv`.

The one-second evidence is central to the thesis. It shows why deployment-oriented evaluation must separate ranking quality, gross signal, and net tradability. A model can be directionally informative and still economically unsuitable under realistic friction assumptions.

5.3. Answer to RQ1: Which Graph Family Performs Best?

The answer to RQ1 is that `base_gnn` performs best overall under the controlled entry-model benchmark.

The strongest evidence comes from the two shared-task regimes. At `5min`, the best model is `base-gnn-conv` with `pnl_sum` = 0.020356 . At `1min`, the best model is again `base-gnn-conv`, with `pnl_sum` = 0.020094 . Both results are positive, and both are obtained by the same family and operator.

At `1sec`, no family produces positive net PnL. This means the high-frequency regime cannot be used to identify a robust deployment winner. Instead, it shows that all families face a cost and turnover barrier under the current entry policy.

The family-level conclusion is therefore conservative but clear. Under fixed targets, fixed features, fixed validation, fixed thresholds, and fixed event-based exits, the simpler single-graph baseline is the most reliable architecture. The richer families may contain useful signal, but they do not produce a stronger post-cost benchmark result.

5.4. Answer to RQ2: How Important Is the Conv-versus-MPNN Operator Choice?

The operator choice is important, but its effect is not universal.

At `5min`, Conv outperforms MPNN on `pnl_sum` in all three families:

1. `base_gnn`: 0.020356 versus 0.006915 .
2. `multigraph`: 0.001958 versus -0.009359 .

3. **memorygraph**: 0.004359 versus -0.037363.

At 1min, Conv remains better for **base_gnn** and **multigraph**, but **memorygraph** reverses in favor of MPNN:

1. **base_gnn**: 0.020094 versus -0.078539.
2. **multigraph**: -0.007833 versus -0.035771.
3. **memorygraph**: -0.078947 versus 0.009305.

At 1sec, all models are net negative, but MPNN is less negative than Conv in each family:

1. **base_gnn**: -0.065821 versus -0.094185.
2. **multigraph**: -0.080723 versus -0.108790.
3. **memorygraph**: -0.280512 versus -1.163268.

The operator therefore changes economic outcomes materially. The strongest shared-task model is Conv-based, but the least negative one-second models are MPNN-based. The correct conclusion is not that Conv is always better or that MPNN is always better. The operator must be selected jointly with the family scaffold, frequency regime, and cost-sensitive trading policy.

5.5. Answer to RQ3: How Does Temporal Resolution Affect Relation and Memory Mechanisms?

The results do not support the hypothesis that finer temporal resolution automatically increases the net economic value of richer graph mechanisms.

For **multigraph**, relation-specific processing does not beat the baseline on **pnL_sum** at any frequency. At five-minute frequency, it is mildly positive in the Conv variant but below the baseline. At one-minute frequency, both variants are net negative. At one-second frequency, both variants have positive gross signal but negative net PnL.

For **memorygraph**, the answer is more nuanced. The family becomes most distinctive at one-second frequency, exactly where stateful memory was expected to matter most. Its gross results are the largest in the benchmark. This provides partial evidence that recurrent memory can surface high-frequency opportunities. However, the same results show that memory also produces excessive trading activity under the current policy. The net effect is strongly negative.

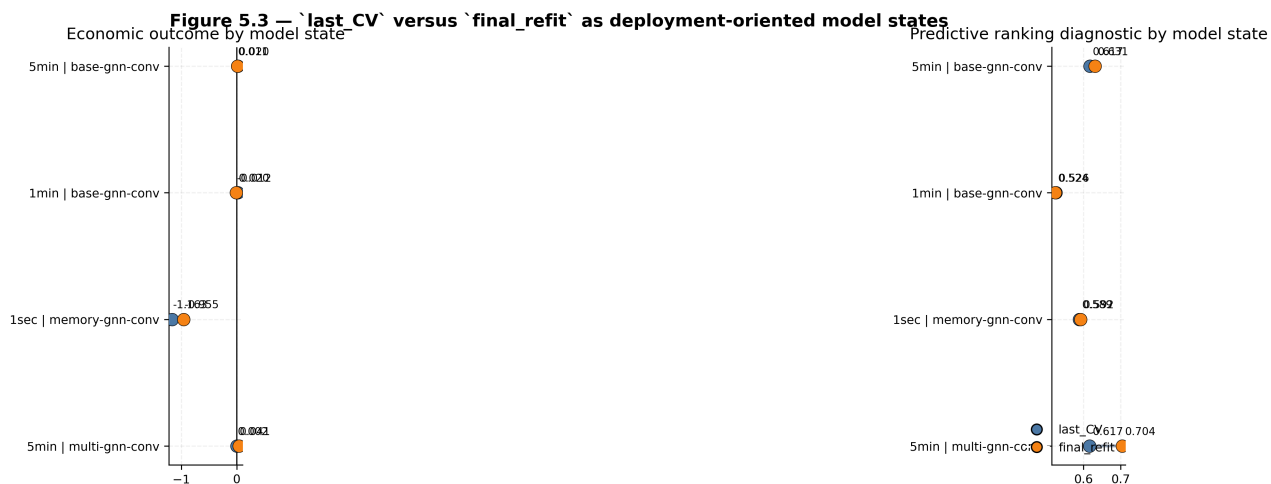
The best interpretation is therefore two-layered. Finer temporal resolution appears to increase the amount of extractable short-horizon signal, especially for memory-based models. At the same time, it increases the penalty for insufficient trade selectivity. In the current benchmark, the cost and turnover effect dominates the signal-extraction effect.

5.6. Answer to RQ4: Are Conclusions Stable Between **last_CV** and **final_refit**?

The deployment-state comparison shows that **last_CV** and **final_refit** are related but not interchangeable. The **last_CV** state remains the main deployment reference because it is produced by the final chronological cross-validation fold. The **final_refit** state is useful because it tests what happens when a model is refit on a larger pre-holdout sample, but it does not replace the chronological evidence.

The conceptual distinction between these states is summarized in Figure 5.3 before the selected numerical comparisons are reported.

Figure 5.3. **last_CV versus **final_refit** as deployment-oriented model states.**



Selected cases mirror the thesis discussion: two best shared-task models, one high-frequency stress example, and one informative multigraph refit case.

Interim submission placeholder: This figure should contrast the latest chronological cross-validation model used as the primary deployment reference with the larger-sample `final_refit` model used as an informative but non-primary comparison.

5.6.1. Best Five-Minute Model: `base-gnn-conv`

Table 5.2. `last_cv` versus `final_refit` for the best five-minute model.

Frequency	Training cycle	gross_pnl_sum	pnl_sum	n_trades	dir_auc	trade_auc
5min	last_cv	0.028156	0.020356	26	0.617105	0.700447
5min	final_refit	0.017570	0.011270	21	0.630702	0.721795

The five-minute winner remains positive after refitting. Its net PnL declines, but its ranking metrics improve. This is the cleanest deployment-stability case in the selected comparisons. It also shows why final refitting should not be assumed to improve the main economic metric: more training data improve AUC here, but not `pnl_sum`.

5.6.2. Best One-Minute Model: `base-gnn-conv`

Table 5.3. `last_cv` versus `final_refit` for the best one-minute model.

Frequency	Training cycle	gross_pnl_sum	pnl_sum	n_trades	dir_auc	trade_auc
1min	last_cv	0.059694	0.020094	132	0.525927	0.648157
1min	final_refit	0.007198	-0.012002	64	0.524286	0.635712

The one-minute winner is less stable. The `last_cv` model is clearly positive, while the `final_refit` version turns negative. The AUC values change only modestly, which suggests that the underlying ranking quality remains similar while the score-to-trade conversion becomes less economically favorable. This case reinforces the deployment argument: a model can look similar in predictive diagnostics but materially different in realized trading performance.

5.6.3. Selected One-Second Memorygraph Case: `memory-gnn-conv`

Table 5.4. `last_cv` versus `final_refit` for the selected one-second memorygraph case.

Frequency	Training cycle	gross_pnl_sum	pnl_sum	n_trades	dir_auc	trade_auc
1sec	last_cv	0.412032	-1.163268	5251	0.588785	0.490050
1sec	final_refit	0.443031	-0.954969	4660	0.592186	0.852874

The selected one-second case is the most informative high-frequency stress example. Refitting increases gross PnL, reduces the trade count, improves `trade_auc`, and makes the net result less negative. Nevertheless, the model

remains strongly unprofitable after costs. The central one-second conclusion therefore survives refitting: memory-based high-frequency signal is present, but it is not sufficiently selective under the current benchmark.

5.6.4. Informative Five-Minute Refit Case: **multi-gnn-conv**

Table 5.5. **last_CV versus **final_refit** for the informative five-minute multigraph case.**

Frequency	Training cycle	gross_pnl_sum	pnl_sum	n_trades	dir_auc	trade_auc
5min	last_CV	0.012758	0.001958	36	0.616667	0.672097
5min	final_refit	0.061167	0.041367	66	0.703947	0.720920

This case is analytically important because the **final_refit** version of **multi-gnn-conv** reaches **dir_auc** > 70% and a strongly positive **pnl_sum**. It shows that relation-preserving graph processing can become highly effective under a larger-sample refit. However, it does not overturn the primary **last_CV** conclusion. In the deployment-oriented state, **multi-gnn-conv** is only weakly positive and remains below **base-gnn-conv**. The case should therefore be interpreted as evidence of refit sensitivity and potential future value, not as proof that **multigraph** is already the strongest deployable family.

5.7. Hypothesis Assessment

H1. The one-minute regime should be the strongest shared-task benchmark.

H1 is not supported on the primary economic metric. The strongest one-minute model reaches **pnl_sum** = **0.020094**, while the strongest five-minute model reaches **pnl_sum** = **0.020356**. The difference is small, but the hypothesis predicts one-minute superiority, which is not observed. The one-minute regime remains important because it produces more trades and much larger gross signal, but it is not the strongest shared-task regime in strict net terms.

H2. Explicit multi-relation modelling should outperform the simpler baseline more clearly at finer resolutions.

H2 is not supported on the primary economic benchmark. **multigraph** does not beat **base_gnn** on **pnl_sum** at five-minute, one-minute, or one-second frequency. The interesting **multi-gnn-conv** final-refit case provides evidence that relation-specific modelling can become strong under some training states, but the deployment-oriented **last_CV** benchmark does not support a general multigraph advantage.

H3. Stateful memory should become more valuable as the market is observed more finely.

H3 is partially supported for gross signal extraction but not supported for net deployable performance. At one-second frequency, **memorygraph** produces the strongest gross PnL values in the study. However, both memory variants remain net negative, and **memory-gnn-conv** is especially negative because of excessive turnover. Memory helps reveal short-lived signal, but the current benchmark does not show that it improves post-cost profitability.

H4. Conv and MPNN operators should not be uniformly dominant across families.

H4 is supported. Conv dominates the five-minute net results and produces the strongest shared-task model overall. At one-minute frequency, Conv remains better for **base_gnn** and **multigraph**, while MPNN is better for **memorygraph**. At one-second frequency, MPNN is less negative in all families on net PnL. The operator effect is therefore economically meaningful and frequency-dependent.

H5. **last_CV** and **final_refit** should tell a broadly consistent family-level story.

H5 is partially supported. The selected comparisons do not overturn the broad conclusion that **base_gnn** is the most reliable deployment-oriented family and that one-second models remain net negative. However, the model-level story can change materially. The one-minute **base-gnn-conv** turns negative after refitting, while the five-minute **multi-gnn-conv** becomes very strong after refitting. The evidence therefore supports reporting both states transparently and prioritizing **last_CV** for deployment interpretation.

5.8. Summary of Research Question Answers

Table 5.6 consolidates the answers to the four research questions. The table is included to make the empirical storyline explicit before the thesis moves from results to discussion.

Table 5.6. Summary of answers to the research questions.

Research question	Main empirical answer	Key evidence	Interpretation
RQ1. Which graph family performs best under a controlled entry-model benchmark?	<code>base_gnn</code> performs best overall.	<code>base-gnn-conv</code> is the strongest <code>last_CV</code> model at both <code>5min</code> and <code>1min</code> , with <code>pnL_sum = 0.020356</code> and <code>pnL_sum = 0.020094</code> , respectively.	The simpler single-graph baseline is the most reliable architecture under the tested benchmark.
RQ2. How important is the Conv-versus-MPNN operator choice?	Operator choice materially changes outcomes, but no operator is universally dominant.	Conv is strongest in the shared-task winners, while MPNN is less negative for all families at <code>1sec</code> .	Operator choice should be evaluated jointly with family scaffold, frequency, and cost-sensitive thresholding.
RQ3. How does temporal resolution affect relation and memory mechanisms?	Higher temporal resolution increases gross signal visibility but also increases turnover and cost pressure.	<code>memory-gnn-conv</code> has the largest <code>1sec</code> gross signal, but it is strongly net negative because it executes 5251 trades.	Finer data can reveal short-lived signal, but selectivity becomes the main bottleneck.
RQ4. Are conclusions stable between <code>last_CV</code> and <code>final_refit</code> ?	The broad story is stable, but model-level economics can change substantially.	<code>5min base-gnn-conv</code> remains positive after refit, <code>1min base-gnn-conv</code> turns negative, and <code>5min multi-gnn-conv</code> becomes very strong after refit.	<code>final_refit</code> is informative, but <code>last_CV</code> remains the primary deployment-oriented reference.

5.9. Summary of Hypothesis Assessment

Table 5.7 summarizes the hypothesis assessment. This summary separates unsupported, partially supported, and supported claims and clarifies which conclusions are based on net economic outcomes rather than only on predictive diagnostics.

Table 5.7. Summary of hypothesis assessment.

Hypothesis	Status	Main reason
H1. The one-minute regime should be the strongest shared-task benchmark.	Not supported	The best one-minute net result (<code>pnL_sum = 0.020094</code>) is slightly below the best five-minute net result (<code>pnL_sum = 0.020356</code>).
H2. Explicit multi-relation modelling should outperform the simpler baseline more clearly at finer resolutions.	Not supported	<code>multigraph</code> does not beat <code>base_gnn</code> on <code>last_CV pnL_sum</code> at <code>5min</code> , <code>1min</code> , or <code>1sec</code> .
H3. Stateful memory should become more valuable as the market is observed more finely.	Partially supported	<code>memorygraph</code> produces the strongest one-second gross signal, but it remains strongly negative after costs.
H4. Conv and MPNN operators should not be uniformly dominant across families.	Supported	Conv dominates the shared-task winners, while MPNN is less negative across all families at <code>1sec</code> .
H5. <code>last_CV</code> and <code>final_refit</code> should tell a broadly consistent family-level story.	Partially supported	The broad deployment conclusion remains cautious, but individual model outcomes change materially after refitting.

6. Discussion

6.1. Main Findings by Research Question

The main finding is that architectural complexity did not guarantee better trading outcomes. Under the strict shared-task benchmark, the simpler `base_gnn` family is strongest, and the winning specification is `base-gnn-conv`. This result is methodologically important because it was obtained under a common target construction, common output interface, common thresholding framework, and common event-based backtest.

For RQ1, the strongest family is `base_gnn`. For RQ2, the operator effect is substantial and conditional, but Conv is the stronger shared-task choice. For RQ3, finer temporal resolution increases the visibility of gross high-frequency signal, especially for memorygraph, but it also increases cost drag and turnover risk. For RQ4, `last_CV` and `final_refit` provide complementary but non-equivalent evidence.

A central theme is that predictive quality, gross signal extraction, and net profitability must be separated. The `1sec` memorygraph results make this most visible. A model can generate a large positive gross PnL and still fail economically because the number of trades is too high. Conversely, a model with less dramatic ranking statistics can be more useful if it is more selective.

6.2. Comparison with Previous Work

The results are consistent with the broader limit order book literature in one respect: short-horizon market data contain learnable structure [3-5]. The presence of positive gross PnL in several models, especially at `1sec`, supports the idea that order book and cross-asset microstructure features contain information about subsequent movement.

However, the results also qualify optimistic interpretations of deep learning for market prediction. Prior work such as DeepLOB emphasizes the ability of neural models to learn from high-dimensional order book states [5]. This thesis shows that, in a graph-based crypto setting, representation learning alone is not sufficient. The economic translation layer matters. Thresholds, trade frequency, costs, and deployment state can change the conclusion even when ranking metrics appear reasonable.

The graph-learning literature motivates the use of relation-aware models, and recent financial graph studies motivate multi-relational dynamic modelling [6-14]. The present findings are more cautious. Multi-relation modelling is plausible and sometimes useful, but in the primary `last_CV` benchmark it does not beat the simpler baseline. This does not contradict the graph literature; rather, it shows that graph complexity must be evaluated against the specific economic objective and data regime.

The temporal graph and memory literature also provides a useful comparison [11-12]. Temporal memory is designed to preserve information across changing graph states. The memorygraph results support this idea before costs: the largest `1sec` gross signals come from memory-based models. Yet they also show that memory without sufficiently selective trading control can amplify turnover. In market microstructure, the ability to detect many short-lived opportunities is not enough if the opportunities are too small relative to costs.

6.3. Scientific Implications

The scientific implication is that graph-based market microstructure research should evaluate architecture under deployment-aware metrics, not only under predictive metrics. AUC, accuracy, and regression error remain useful, but they do not settle whether a model is economically meaningful. The benchmark must include trade count, gross PnL, net PnL, and cost sensitivity.

A second implication is that relation modelling should be treated as an empirical design choice rather than an assumed improvement. Preserving multiple relation channels is theoretically attractive, especially in finance, but the updated benchmark shows that early fusion in a simpler baseline can outperform late relation-specific processing under some conditions.

A third implication concerns temporal resolution. Higher frequency can increase the quantity of detectable signal, but it also raises the cost of acting on that signal. Research that evaluates high-frequency models without explicit turnover and cost analysis risks overstating practical value.

6.4. Practical and Deployment Implications

The strongest deployment-oriented evidence favors **base-gnn-conv** in the shared-task regimes. Even this conclusion should be interpreted cautiously. The model is not a production trading system; it is the best entry model in a controlled final-holdout benchmark. A real deployment would require latency modelling, exchange-specific fee and slippage assumptions, monitoring for regime drift, capital constraints, live order placement logic, and risk controls.

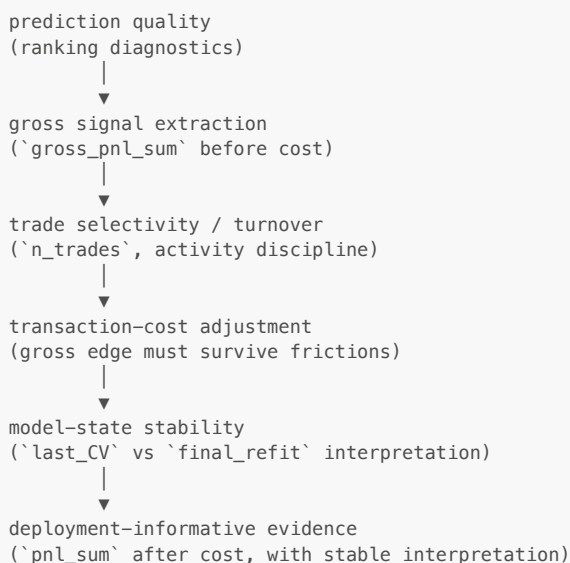
last_CV is deployment-relevant because it resembles the chronological situation of using the latest available model on an unseen future segment. **final_refit** adds information about whether a larger training sample changes the holdout result, but it cannot replace **last_CV**. The **1min** example demonstrates why: the refit model preserves similar ranking diagnostics but turns net negative. The **5min** multigraph Conv example demonstrates the opposite possibility: a model that is weak in **last_CV** can become strong under final refit. Both cases show that deployment interpretation depends on model state.

The updated results imply that realistic deployment should prioritize selectivity and cost robustness. The high-frequency memory models are not deployable in their current form because their gross edge is overwhelmed by turnover. Future deployment-oriented work should therefore evaluate trade-rate controls, threshold stability, cost sensitivity, and no-trade calibration before treating high-frequency graph signals as practically useful.

Figure 6.1 summarizes the practical interpretation chain that connects predictions to deployment-oriented evidence.

Figure 6.1. Deployment interpretation from prediction to post-cost evidence.

Interim submission placeholder: This figure should show that a model must pass through predictive ranking, gross signal extraction, trade selectivity, transaction-cost adjustment, and model-state stability before it can be considered deployment-informative.



6.5. Limitations, Weaknesses, and Sources of Bias

Several limitations affect the interpretation of the thesis. They do not invalidate the benchmark, but they define the scope within which the conclusions are valid.

First, the asset universe is small. The graph contains only ADA, BTC, and ETH, with ETH as the sole target asset. This improves interpretability and keeps the architecture comparison controlled, but it creates asset-selection bias. The results may not transfer to larger crypto universes, equities, futures, foreign exchange, or less liquid instruments. A larger universe could also change the value of **multigraph**, because more assets would create more relation pathways.

Second, the data source is a public Kaggle dataset rather than a proprietary exchange feed. The order book snapshots are suitable for a thesis benchmark, but they are not equivalent to full message-level exchange data. They do not

contain all order submissions, cancellations, queue-position information, partial fills, latency measurements, or execution reports. This snapshot-based design limits what can be concluded about live execution.

Third, the benchmark may contain market-regime and temporal-slice bias. A chronological final holdout is more realistic than random splitting, but it still represents a particular late segment of the available sample. If this interval has unusual volatility, liquidity, or directional behavior, the measured ranking of architectures may partly reflect that regime. A broader study would repeat the benchmark across multiple calendar periods, volatility regimes, and market states.

Fourth, the label construction introduces label-construction bias. Triple-barrier labels depend on barrier widths, rolling volatility estimates, timeout treatment, and the decision to mask some direction labels. These choices are defensible and shared across models, but they shape what the models learn. A different barrier system could change the apparent value of relation-preserving or memory-based architectures.

Fifth, threshold-selection bias remains possible. Trade and direction thresholds are selected from finite validation grids and then applied to the final holdout. This is more disciplined than tuning on the holdout itself, but realized trading performance still depends on the threshold grid, feasibility constraints, and no-trade calibration. The one-second results indicate that threshold design is one of the main unresolved weaknesses of the benchmark.

Sixth, the transaction-cost model is simplified. A constant round-trip proxy is useful for controlled comparison, but real trading costs include exchange fees, bid-ask spread, queue priority, slippage, market impact, latency, adverse selection, and failed execution. The high-frequency conclusions are especially sensitive to this limitation because costs dominate the one-second outcomes.

Seventh, the model-family set is limited. The thesis compares three graph families and two graph-operator styles. It does not test transformer-based LOB models, hybrid attention-GNN architectures, reinforcement-learning exits, probabilistic calibration layers, or explicitly turnover-regularized objectives. The conclusion is therefore conditional on the tested model families.

Eighth, hyperparameter search is limited. The benchmark is designed for fair comparison, not exhaustive optimization. Some architectures, especially `multigraph` and `memorygraph`, might improve under broader tuning. The informative five-minute `multi-gnn-conv final_refit` case suggests that richer models may be sensitive to training state and calibration.

Ninth, the `1sec` regime is frequency-adapted rather than directly equivalent to the slower regimes. It uses a shorter clock-time task, a restricted working sample, fewer cross-validation folds, and an enlarged holdout fraction for feasibility and alignment. The `1sec` results should therefore be interpreted as a high-frequency stress test rather than as a perfectly symmetric extension of the `5min` and `1min` shared task.

Tenth, deployment remains conceptual rather than operationally complete. The backtest is sequential and event-based, but it is not a live trading system. It does not include live data ingestion, order placement, monitoring, risk limits, capital allocation, latency measurement, operational failure handling, or compliance controls. The thesis can support deployment-oriented conclusions, but not live-profitability claims.

Finally, model-selection bias is a general risk in empirical machine learning. Even when the final holdout is protected, repeated experimentation can indirectly shape modelling choices. Reporting both `last_cv` and `final_refit`, preserving the full benchmark table, and emphasizing limitations reduces this risk, but cannot eliminate it completely.

7. Conclusions and Future Research

7.1. Overall Conclusion

This thesis asked whether graph-based and memory-aware architectures improve short-horizon limit order book prediction under a controlled, deployment-oriented benchmark. The updated empirical answer is conservative. The strongest `last_cv` evidence favors the simpler `base_gnn` family, specifically `base-gnn-conv`, at both shared-task frequencies.

At five-minute frequency, `base-gnn-conv` achieves the best `last_cv` net result with `pnl_sum = 0.020356` over 26 trades. At one-minute frequency, `base-gnn-conv` again achieves the best result with `pnl_sum = 0.020094` over 132 trades. At one-second frequency, all models are net negative after transaction costs. The core high-frequency

finding is therefore the divergence between gross signal and net deployability. `memory-gnn-conv` produces the largest one-second gross signal, but it fails after costs because the trading policy expresses that signal through excessive turnover.

The thesis does not show that graph neural networks are ineffective for market microstructure. It shows something more precise: under a fair entry-model benchmark, additional relation-specific processing and recurrent memory do not automatically produce better post-cost trading performance. The strongest architecture is the one that best balances signal extraction, selectivity, and turnover.

The deployment-state analysis reinforces this conclusion. `last_cv` should remain the primary deployment reference because it respects chronological model selection. `final_refit` is useful but can alter economic outcomes in both directions. The five-minute baseline refit remains positive, the one-minute baseline refit turns negative, and the five-minute `multi-gnn-conv` refit becomes very strong. These cases show why both states should be reported transparently and why `final_refit` should not replace chronological deployment evidence.

The final thesis conclusion is therefore disciplined rather than promotional: richer graph architectures extract useful signals in some cases, but they do not establish a robust post-cost advantage over the simpler `base_gnn` benchmark under the tested dataset, model families, and evaluation protocol. The main unresolved challenge is not merely extracting short-horizon microstructure signal. It is converting that signal into sufficiently selective, stable, and cost-aware trading decisions.

7.2. Future Research

Future research should focus first on turnover-aware modelling. The `1sec` experiments show that memory-based graph models can identify many short-lived opportunities, but they lack sufficient selectivity. Future work should investigate cost-aware objectives, stricter no-trade calibration, sparse event-driven state updates, confidence-aware memory resets, and threshold policies that explicitly penalize excessive trading.

A second direction is execution-aware evaluation. The present benchmark fixes a common realized-event exit rule in order to preserve fairness. Future work could keep the common entry benchmark for comparability and then test the strongest entry models under adaptive exits, richer slippage assumptions, exchange-specific fees, latency constraints, queue-position modelling, partial-fill assumptions, and adverse-selection scenarios.

A third direction is larger-universe graph modelling. A three-asset graph is useful for a controlled thesis benchmark, but it may understate the value of relation-specific architectures. Adding more crypto assets, stablecoins, sector proxies, derivatives, or cross-venue liquidity measures would create a stronger test of whether `multigraph` becomes more valuable when the relation space is richer.

A fourth direction is selective memory. The current `memorygraph` models appear capable of finding high-frequency gross signal but not of controlling trade frequency. Future models should examine sparse memory updates, event-triggered memory writes, confidence-aware state resets, and memory mechanisms coupled to explicit trade-rate constraints.

A fifth direction is robustness and uncertainty analysis. Future versions of the benchmark should report fold-level dispersion, bootstrap confidence intervals for economic metrics, pairwise model-comparison tests, regime-specific performance summaries, drawdown statistics, and cost-sensitivity curves. These additions would make it easier to distinguish genuine architecture effects from temporal-slice effects or threshold-selection effects.

A final direction is systematic cost sensitivity. Because the main high-frequency weakness is the gap between gross and net performance, future research should report cost-sensitivity curves across fee, spread, slippage, and latency assumptions. This would clarify whether a model is close to viability under realistic execution improvements or whether its signal is too small relative to unavoidable trading frictions.

The future research agenda is summarized in Figure 7.1.

Figure 7.1. Future research roadmap for deployment-oriented graph LOB prediction.

Interim submission placeholder: This figure should organize future work into turnover-aware learning, execution-aware evaluation, larger graph universes, selective memory mechanisms, regime robustness, uncertainty quantification, and cost-sensitivity analysis.

Appendix A. Model Configuration Summary

This appendix summarizes the main benchmark configuration in a compact form. It is not intended to replace the Methodology chapter; instead, it provides a quick reference for the model-family, operator, frequency, and target-design choices used throughout the thesis.

Table A.1. Model-family and operator mapping.

Benchmark label	Family	Graph operator	Main architectural distinction
base-gnn-conv	base_gnn	adaptive_conv	Early relation fusion with Conv-style graph processing.
base-gnn-mpnn	base_gnn	adaptive_mpnn	Early relation fusion with MPNN-style message passing.
multi-gnn-conv	multigraph	dynamic_rel_conv	Relation-specific graph pathways with Conv-style processing.
multi-gnn-mpnn	multigraph	dynamic_edge_mpnn	Relation-specific graph pathways with MPNN-style processing.
memory-gnn-conv	memorygraph	conv	Stateful recurrent node-edge memory with Conv-style graph updates.
memory-gnn-mpnn	memorygraph	mpnn	Stateful recurrent node-edge memory with MPNN-style graph updates.

Table A.2. Frequency and task configuration.

Frequency	Lookback	Forecast horizon	Working data slice	Final holdout fraction	CV folds	Interpretation
5min	30 minutes = 6 bars	5 minutes = 1 bar	0.0–0.9 of the full series	0.10	4	Strict shared-task benchmark.
1min	30 minutes = 30 bars	5 minutes = 5 bars	0.0–0.9 of the full series	0.10	4	Strict shared-task benchmark.
1sec	2 minutes = 120 bars	2 minutes = 120 bars	0.5–0.9 of the full series	0.225	2	Frequency-adapted high-frequency stress test.

Table A.3. Shared target and loss configuration.

Component	Configuration used in the benchmark
Target asset	ETH
Context assets	ADA and BTC
Target construction	Volatility-scaled triple-barrier framework
Output heads	trade_logit, dir_logit, return_pred, exit_type_logit, tte_pred
Loss weights	loss_w_trade = 0.35, loss_w_dir = 0.65, loss_w_ret = 0.15, loss_w_utility = 0.85, loss_w_exit_type = 0.05, loss_w_tte = 0.03
Main economic metric	pnl_sum
Cost proxy	c_rt = 0.0003 with cost_bps_per_side = 1.0

Appendix B. Additional Benchmark Tables

This appendix records compact supplementary summaries that may be useful for a reader who wants to compare the main deployment-oriented result with selected robustness cases. The complete `last_cv` benchmark is reported in Table 5.1, while selected `last_cv` versus `final_refit` comparisons are reported in Tables 5.2-5.5.

Table B.1. Best or least-negative `last_cv` model by frequency.

Frequency	Best or least-negative model	<code>gross_pnl_sum</code>	<code>pnl_sum</code>	<code>n_trades</code>	Interpretation
5min	base-gnn-conv	0.028156	0.020356	26	Best positive shared-task result.
1min	base-gnn-conv	0.059694	0.020094	132	Best one-minute result, with larger gross signal but higher turnover.
1sec	base-gnn-mpnn	0.052679	-0.065821	395	Least negative one-second model; still not deployment-grade after costs.

Table B.2. Main high-frequency turnover example.

Model	Frequency	<code>gross_pnl_sum</code>	<code>pnl_sum</code>	<code>n_trades</code>	Approximate cumulative cost	Interpretation
memory-gnn-conv	1sec	0.412032	-1.163268	5251	1.5753	Strong gross signal is overwhelmed by excessive turnover and transaction costs.

Appendix C. Final Holdout Alignment Table

Source: `final_runs/*/splits/split_summary.json`, `split_indices.npz`, `resolved_config.yaml`.

Interpretation: - `literal_match_within_freq = True` means the saved `idx_holdout` arrays are exactly identical within that frequency regime. - `final_test_equals_holdout = True` means `final_production.test` is exactly the same saved holdout interval. - `semantic_alignment_vs_1min` compares calendar-time alignment against the 1-minute reference window.

run	freq	slice	holdout_frac	holdout_start_utc	holdout_end_utc	holdout_n	final_test_equals_h
1min-base-gnn-conv	1min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	True
1min-base-gnn-mpnn	1min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	True
1min-memory-gnn	1min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	True
1min-multi-gnn-conv	1min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	True

run	freq	slice	holdout_frac	holdout_start_utc	holdout_end_utc	holdout_n	final_test_equals_h
1min-multi-gnn-mpnn	1min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	True
1sec-base-gnn-conv	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	True
1sec-base-gnn-mpnn	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	True
1sec-memory-gnn-conv	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	True
1sec-memory-gnn-mpnn	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	True
1sec-multi-gnn-conv	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	True
1sec-multi-gnn-mpnn	1sec	0.5-0.9	0.225	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	True
5min-base-gnn	5min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:05:00	309	True
5min-memory-gnn	5min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:05:00	309	True
5min-multi-gnn	5min	0.0-0.9	0.100	2021-04-17T03:25:00	2021-04-18T05:05:00	309	True

Frequency-level summary

freq	runs	shared_holdout_start_utc	shared_holdout_end_utc	shared_holdout_n	note
1min	5	2021-04-17T03:25:00	2021-04-18T05:07:00	1543	reference window
1sec	6	2021-04-17T03:25:34	2021-04-18T05:10:01	92668	starts 34s later than 1min, ends 181s later
5min	3	2021-04-17T03:25:00	2021-04-18T05:05:00	309	same start as 1min, ends 120s earlier

References

- [1] Cont, R. (2001). *Empirical properties of asset returns: stylized facts and statistical issues*. Quantitative Finance, 1(2), 223-236. PDF: <https://www.stat.rice.edu/~dobelman/courses/texts/stylized.cont.2001.pdf>
- [2] Cont, R., Stoikov, S., & Talreja, R. (2010). *A stochastic model for order book dynamics*. Operations Research, 58(3), 549-563. PDF: <https://rama.cont.perso.math.cnrs.fr/pdf/CST2010.pdf>
- [3] Ntakaris, A., Magris, M., Kannianen, J., Gabbouj, M., & Iosifidis, A. (2018). *Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods*. Journal of Forecasting, 37(8), 852-866. arXiv/PDF: <https://arxiv.org/abs/1705.03233>
- [4] Sirignano, J., & Cont, R. (2019). *Universal features of price formation in financial markets: perspectives from deep learning*. Quantitative Finance, 19(9), 1449-1459. arXiv/PDF: <https://arxiv.org/abs/1803.06917>
- [5] Zhang, Z., Zohren, S., & Roberts, S. (2019). *DeepLOB: Deep convolutional neural networks for limit order books*. IEEE Transactions on Signal Processing, 67(11), 3001-3012. PDF: <https://www.oxford-man.ox.ac.uk/wp-content/uploads/2020/03/DeepLOB-Deep-Convolutional-Neural-Networks-for-Limit-Order-Books.pdf>
- [6] Kipf, T. N., & Welling, M. (2017). *Semi-supervised classification with graph convolutional networks*. International Conference on Learning Representations. arXiv/PDF: <https://arxiv.org/abs/1609.02907>
- [7] Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). *Neural message passing for quantum chemistry*. Proceedings of the 34th International Conference on Machine Learning, PMLR 70, 1263-1272. PDF: <https://proceedings.mlr.press/v70/gilmer17a/gilmer17a.pdf>
- [8] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). *Graph attention networks*. International Conference on Learning Representations. arXiv/PDF: <https://arxiv.org/abs/1710.10903>
- [9] Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C. and Yu, P.S., 2020. A comprehensive survey on graph neural networks. IEEE transactions on neural networks and learning systems, 32(1), pp.4-24. arXiv/PDF: <https://arxiv.org/abs/1901.00596>
- [10] Wu, Z., Pan, S., Long, G., Jiang, J., Chang, X., & Zhang, C. (2020). *Connecting the dots: Multivariate time series forecasting with graph neural networks*. Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 753-763. arXiv/PDF: <https://arxiv.org/abs/2005.11650>
- [11] Kazemi, S. M., Goel, R., Jain, K., Kobyzev, I., Sethi, A., Forsyth, P., & Poupart, P. (2020). *Representation learning for dynamic graphs: A survey*. Journal of Machine Learning Research, 21(70), 1-73. PDF: <https://jmlr.csail.mit.edu/papers/volume21/19-447/19-447.pdf>
- [12] Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., & Bronstein, M. (2020). *Temporal graph networks for deep learning on dynamic graphs*. arXiv preprint arXiv:2006.10637. arXiv/PDF: <https://arxiv.org/abs/2006.10637>
- [13] Wang, J., Zhang, S., Xiao, Y., & Song, R. (2022). *A review on graph neural network methods in financial applications*. Journal of Data Science, 20(2), 111-134. PDF: <https://jds-online.org/journal/JDS/article/1279/file/pdf>
- [14] Qian, H., Zhou, H., Zhao, Q., Chen, H., Yao, H., Wang, J., Liu, Z., Yu, F., Zhang, Z., & Zhou, J. (2024). *MDGNN: Multi-relational dynamic graph neural network for comprehensive and dynamic stock investment prediction*. Proceedings of the AAAI Conference on Artificial Intelligence, 38(13), 14642-14650. arXiv/PDF: <https://arxiv.org/abs/2402.06633>
- [15] Martinsn. *High-Frequency Crypto Limit Order Book Data*. Kaggle dataset. Dataset page: <https://www.kaggle.com/datasets/martinsn/high-frequency-crypto-limit-order-book-data>